



处理机调度

层次

目标

算法

多处理机调度





层次

目标

算法

多处理机调度

高级调度

又称长程调度或作业调度，调度对象是**作业**

根据一定的算法决定将外存上处于后备队列中的哪几个作业调入内存

为它们创建进程、分配必要资源，并将他们放入就绪队列

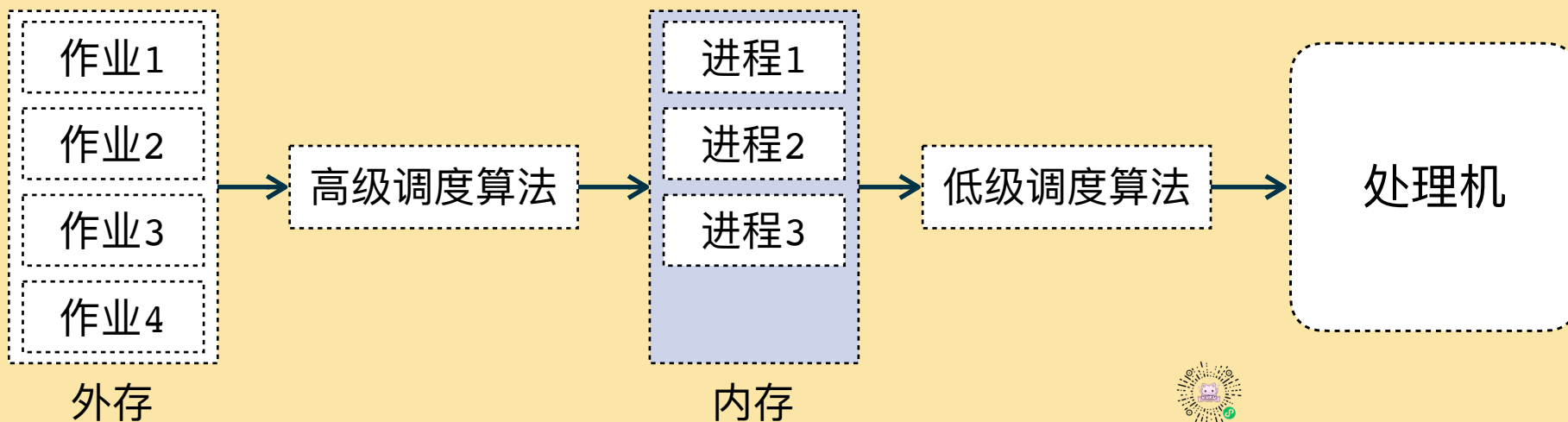
高级调度主要用于多道批处理系统中，分时系统和实时系统中不设置高级调度

低级调度

又称短程调度或进程调度，调度对象是**进程**

根据某种算法决定就绪队列中的哪个进程获得处理机，并由分派程序将处理机分配给被选中进程

低级调度是最基本的一种调度，在多道批处理、分时、实时系统中都必须配置这种调度





层次

目标

算法

多处理机调度

高级调度

又称长程调度或作业调度，调度对象是**作业**

根据一定的算法决定将外存上处于后备队列中的哪几个作业调入内存

为它们创建进程、分配必要资源，并将他们放入就绪队列

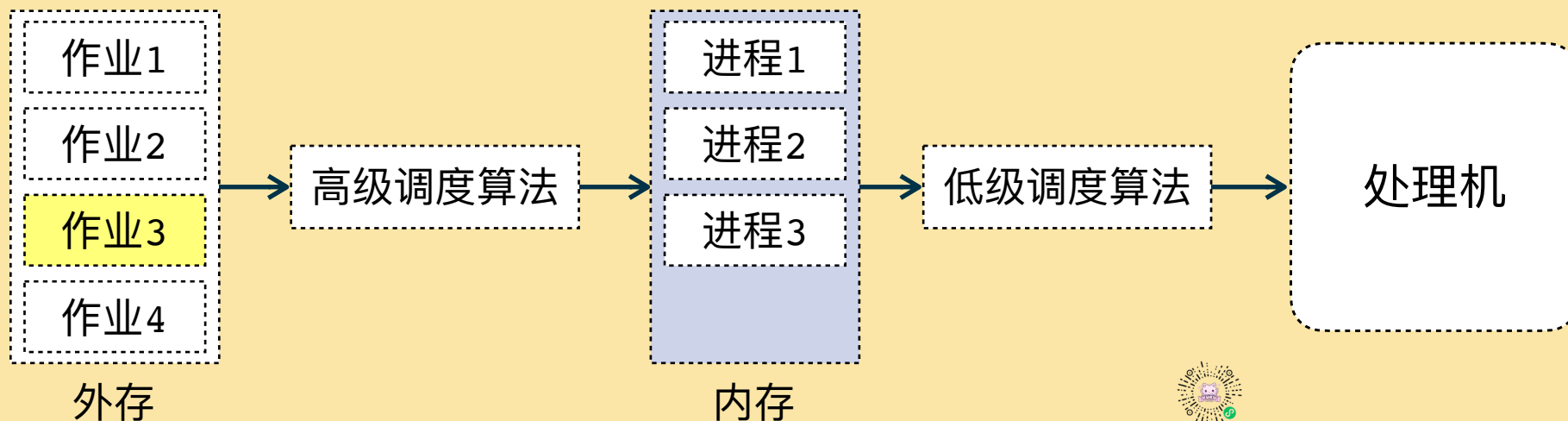
高级调度主要用于多道批处理系统中，分时系统和实时系统中不设置高级调度

低级调度

又称短程调度或进程调度，调度对象是**进程**

根据某种算法决定就绪队列中的哪个进程获得处理机，并由分派程序将处理机分配给被选中进程

低级调度是最基本的一种调度，在多道批处理、分时、实时系统中都必须配置这种调度





层次

目标

算法

多处理机调度

高级调度

又称长程调度或作业调度，调度对象是**作业**

根据一定的算法决定将外存上处于后备队列中的哪几个作业调入内存

为它们创建进程、分配必要资源，并将他们放入就绪队列

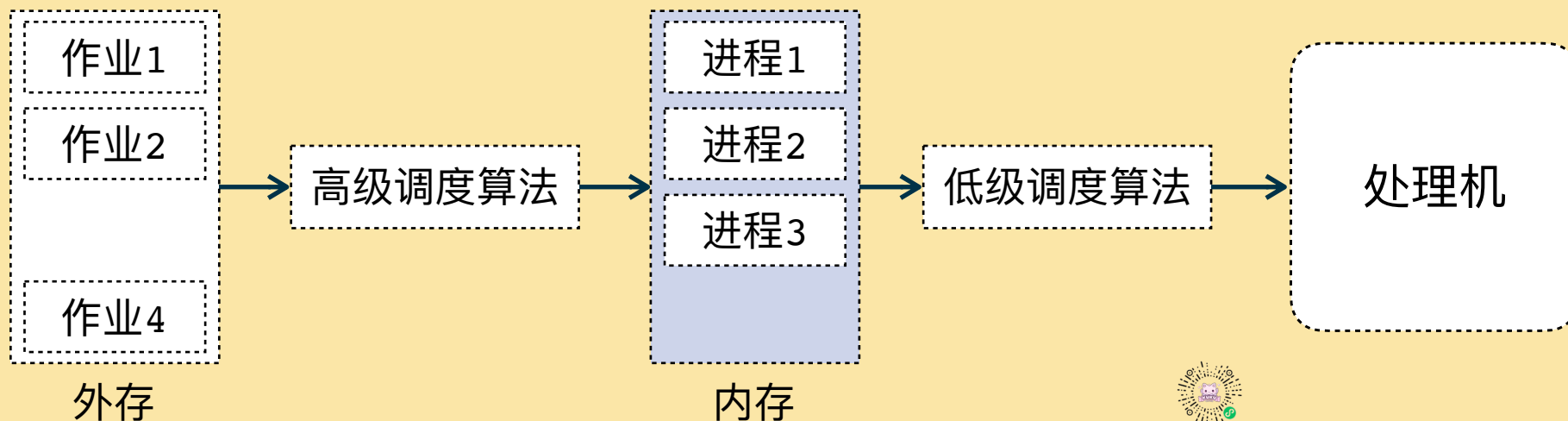
高级调度主要用于多道批处理系统中，分时系统和实时系统中不设置高级调度

低级调度

又称短程调度或进程调度，调度对象是**进程**

根据某种算法决定就绪队列中的哪个进程获得处理机，并由分派程序将处理机分配给被选中进程

低级调度是最基本的一种调度，在多道批处理、分时、实时系统中都必须配置这种调度





层次

目标

算法

多处理机调度

高级调度

又称长程调度或作业调度，调度对象是**作业**

根据一定的算法决定将外存上处于后备队列中的哪几个作业调入内存

为它们创建进程、分配必要资源，并将他们放入就绪队列

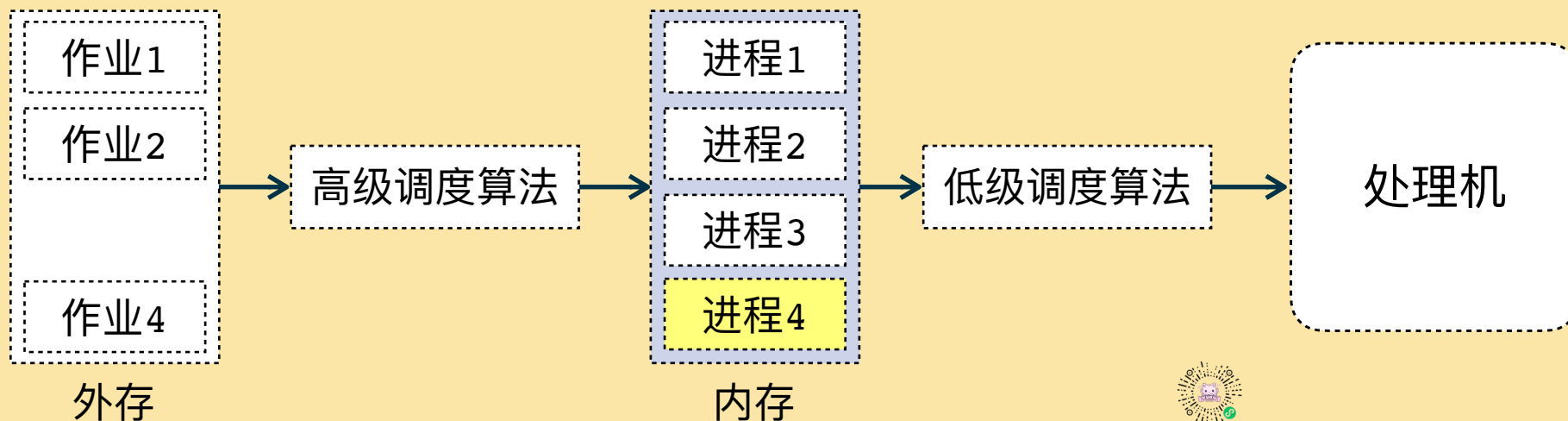
高级调度主要用于多道批处理系统中，分时系统和实时系统中不设置高级调度

低级调度

又称短程调度或进程调度，调度对象是**进程**

根据某种算法决定就绪队列中的哪个进程获得处理机，并由分派程序将处理机分配给被选中进程

低级调度是最基本的一种调度，在多道批处理、分时、实时系统中都必须配置这种调度





层次

目标

算法

多处理机调度

高级调度

又称长程调度或作业调度，调度对象是**作业**

根据一定的算法决定将外存上处于后备队列中的哪几个作业调入内存

为它们创建进程、分配必要资源，并将他们放入就绪队列

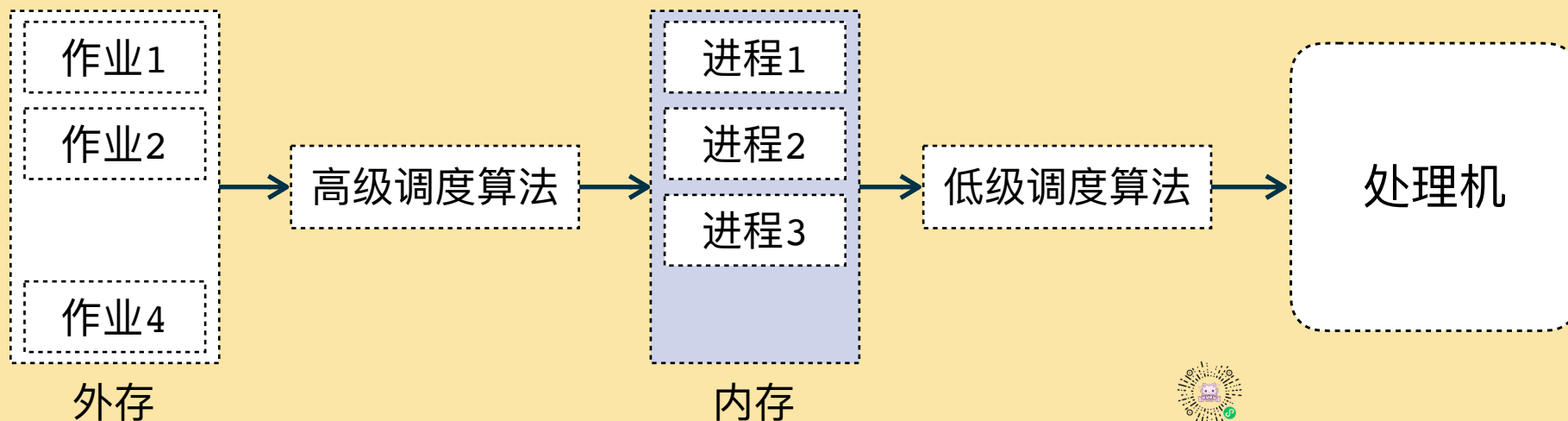
高级调度主要用于多道批处理系统中，分时系统和实时系统中不设置高级调度

低级调度

又称短程调度或进程调度，调度对象是**进程**

根据某种算法决定就绪队列中的哪个进程获得处理机，并由分派程序将处理机分配给被选中进程

低级调度是最基本的一种调度，在多道批处理、分时、实时系统中都必须配置这种调度





层次

目标

算法

多处理机调度

高级调度

又称长程调度或作业调度，调度对象是**作业**

根据一定的算法决定将外存上处于后备队列中的哪几个作业调入内存

为它们创建进程、分配必要资源，并将他们放入就绪队列

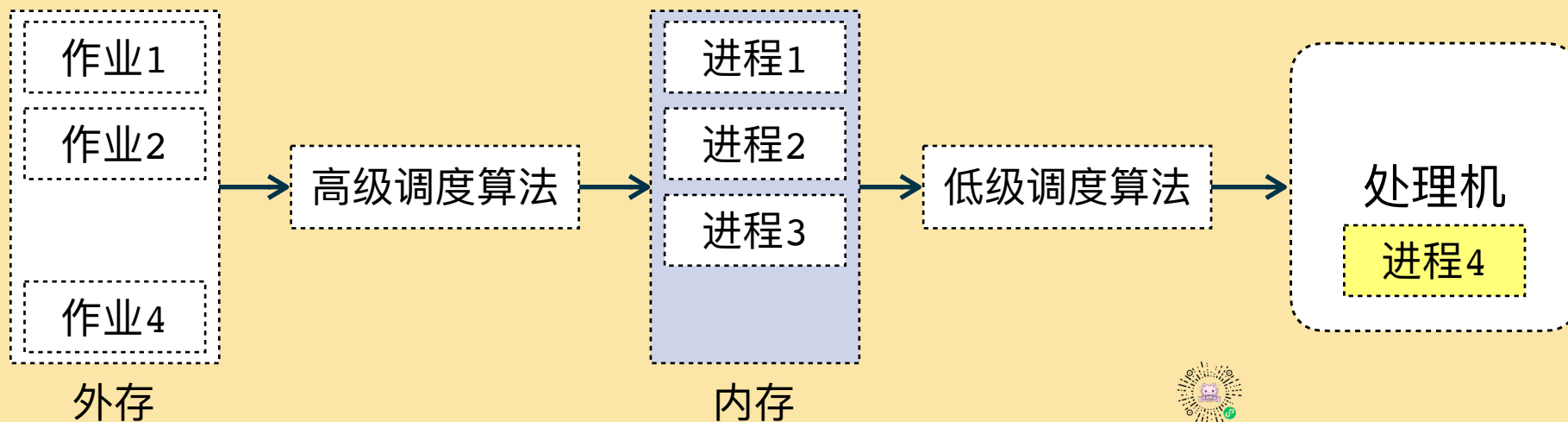
高级调度主要用于多道批处理系统中，分时系统和实时系统中不设置高级调度

低级调度

又称短程调度或进程调度，调度对象是**进程**

根据某种算法决定就绪队列中的哪个进程获得处理机，并由分派程序将处理机分配给被选中进程

低级调度是最基本的一种调度，在多道批处理、分时、实时系统中都必须配置这种调度





层次

目标

算法

多处理机调度

中级调度

又称为内存调度，作用是把那些暂时不能运行的进程调到外存等待

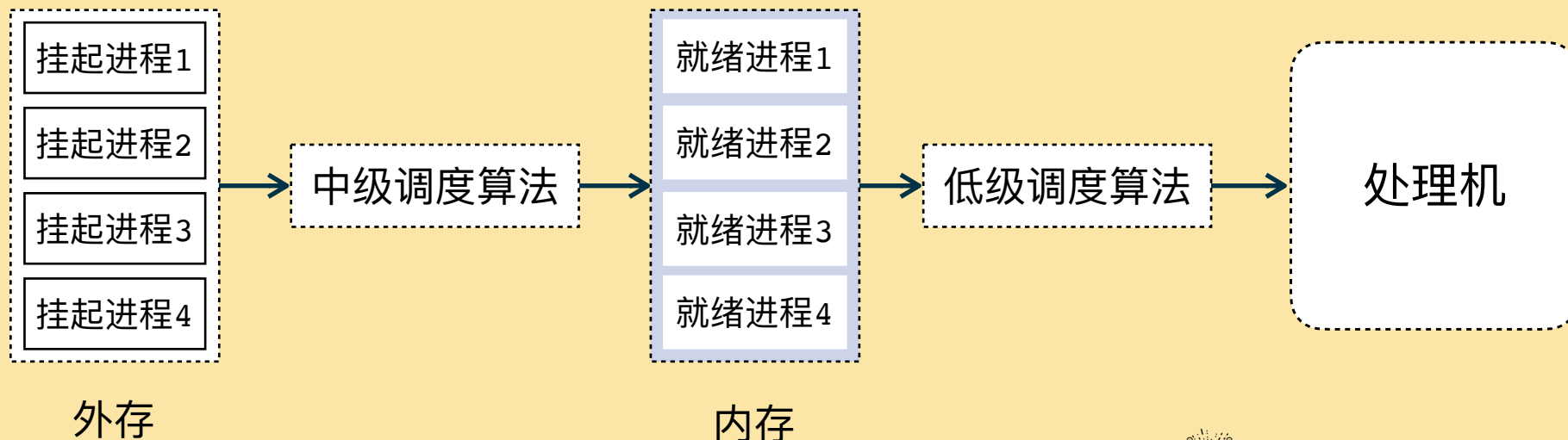
此时进程的状态称为**挂起态**（也叫就绪驻外存状态）

当它们已具备运行条件且内存稍有空闲时

由中级调度来决定把外存上的那些已具备运行条件的就绪进程再次调入内存

并修改它们的状态为就绪状态，挂在就绪队列上等待

中级调度实际上就是存储器管理中的对换功能（后续会介绍）





层次

目标

算法

多处理机调度

中级调度

又称为内存调度，作用是把那些暂时不能运行的进程调到外存等待

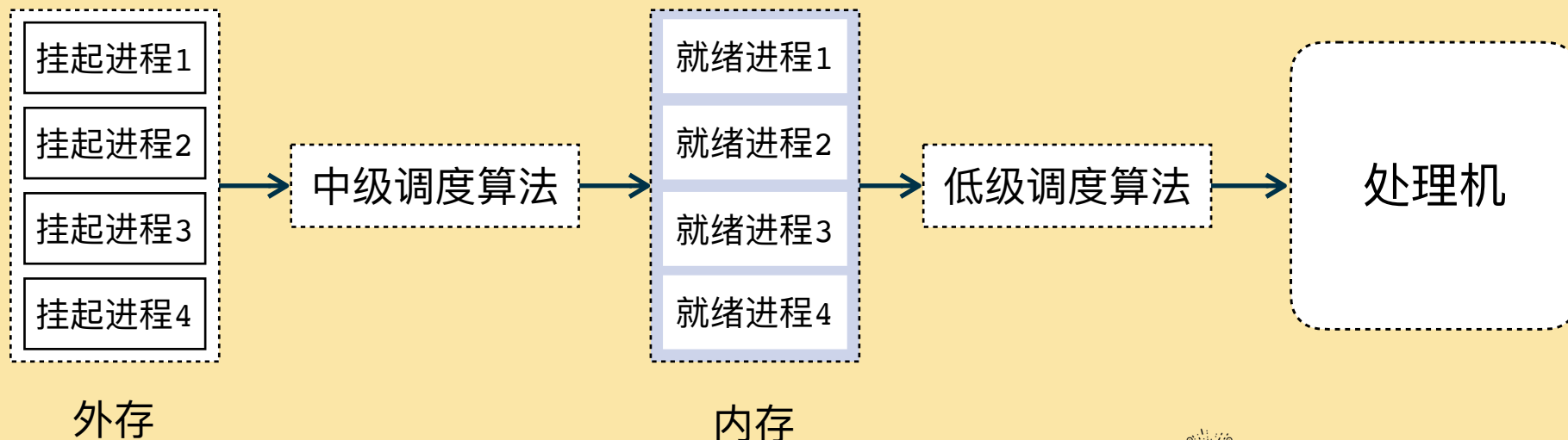
此时进程的状态称为**挂起态**（也叫就绪驻外存状态）

当它们已具备运行条件且内存稍有空闲时

由中级调度来决定把外存上的那些已具备运行条件的就绪进程再次调入内存

并修改它们的状态为就绪状态，挂在就绪队列上等待

中级调度实际上就是存储器管理中的对换功能（后续会介绍）





层次

目标

算法

多处理机调度

中级调度

又称为内存调度，作用是把那些暂时不能运行的进程调到外存等待

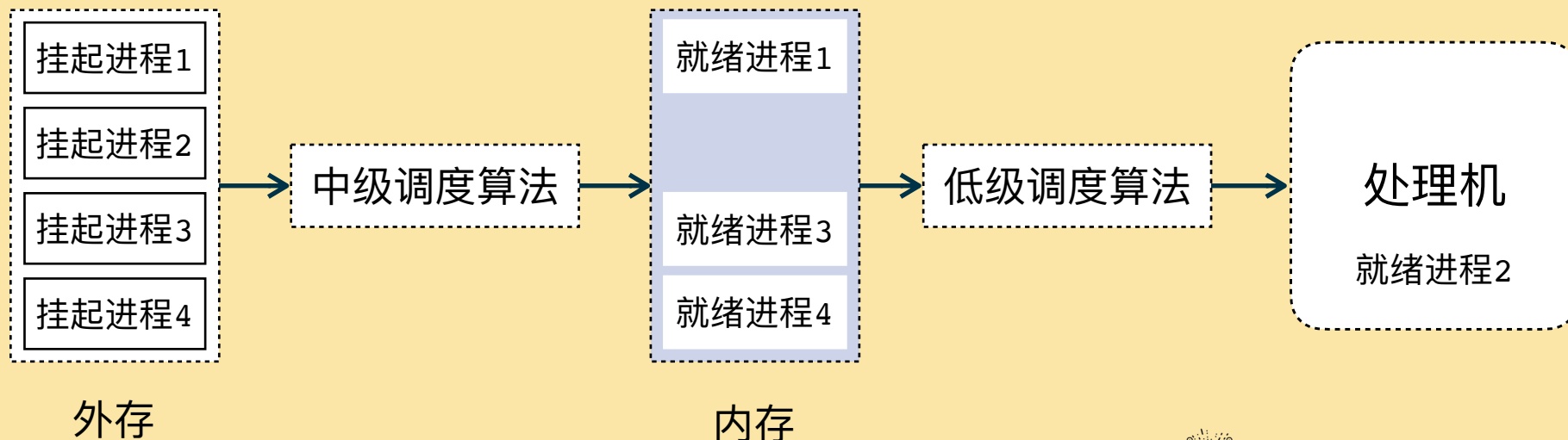
此时进程的状态称为**挂起态**（也叫就绪驻外存状态）

当它们已具备运行条件且内存稍有空闲时

由中级调度来决定把外存上的那些已具备运行条件的就绪进程再次调入内存

并修改它们的状态为就绪状态，挂在就绪队列上等待

中级调度实际上就是存储器管理中的对换功能（后续会介绍）





层次

目标

算法

多处理机调度

中级调度

又称为内存调度，作用是把那些暂时不能运行的进程调到外存等待

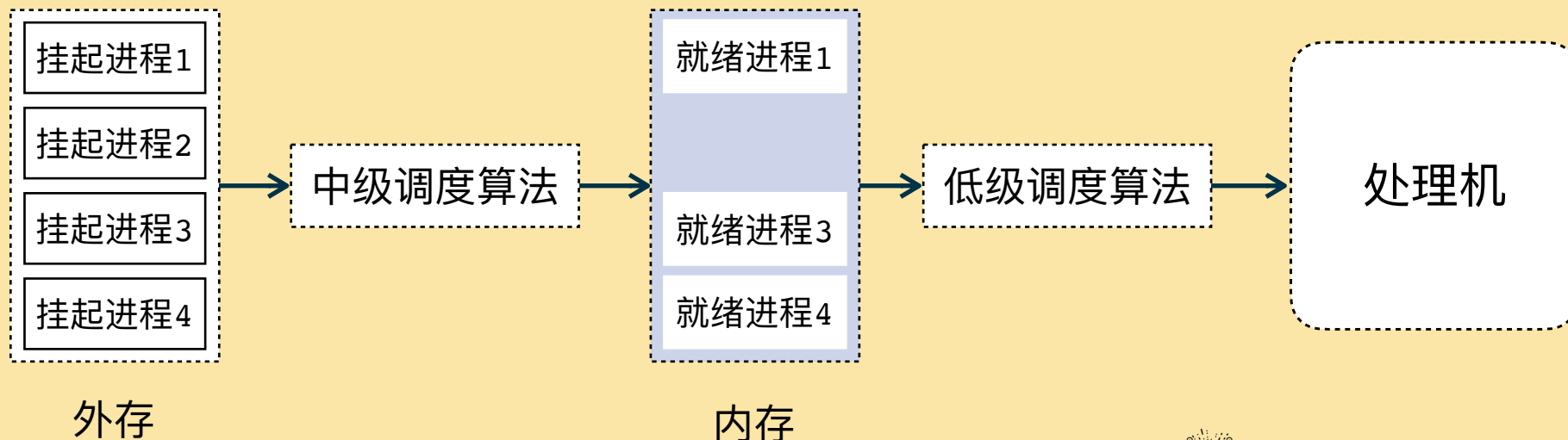
此时进程的状态称为**挂起态**（也叫就绪驻外存状态）

当它们已具备运行条件且内存稍有空闲时

由中级调度来决定把外存上的那些已具备运行条件的就绪进程再次调入内存

并修改它们的状态为就绪状态，挂在就绪队列上等待

中级调度实际上就是存储器管理中的对换功能（后续会介绍）





层次

目标

算法

多处理机调度

中级调度

又称为内存调度，作用是把那些暂时不能运行的进程调到外存等待

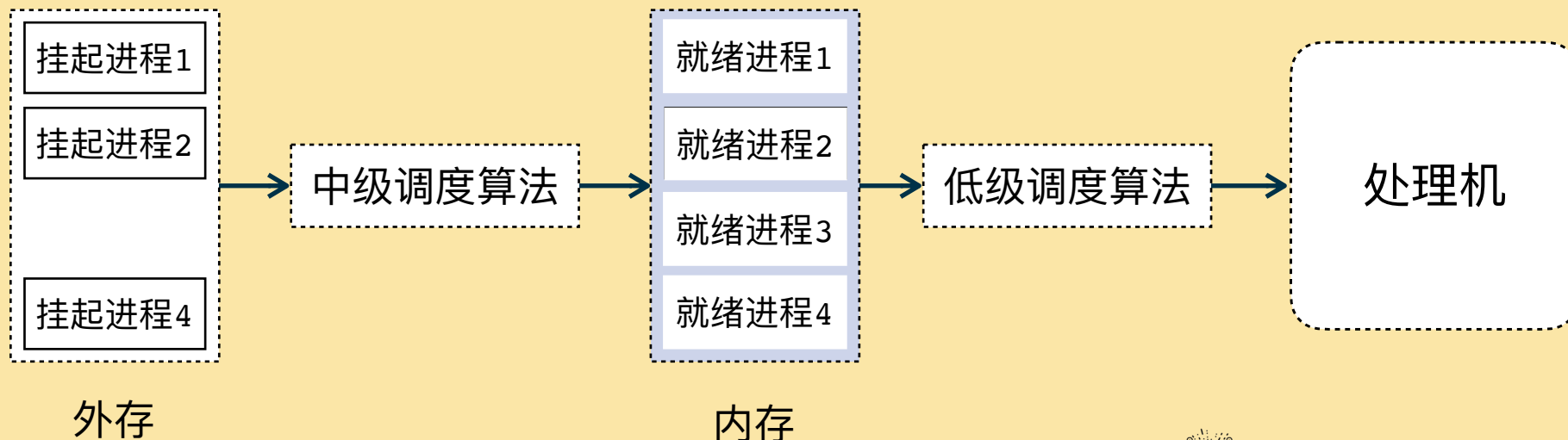
此时进程的状态称为**挂起态**（也叫就绪驻外存状态）

当它们已具备运行条件且内存稍有空闲时

由中级调度来决定把外存上的那些已具备运行条件的就绪进程再次调入内存

并修改它们的状态为就绪状态，挂在就绪队列上等待

中级调度实际上就是存储器管理中的对换功能（后续会介绍）





层次

目标

算法

多处理机调度

中级调度

又称为内存调度，作用是把那些暂时不能运行的进程调到外存等待

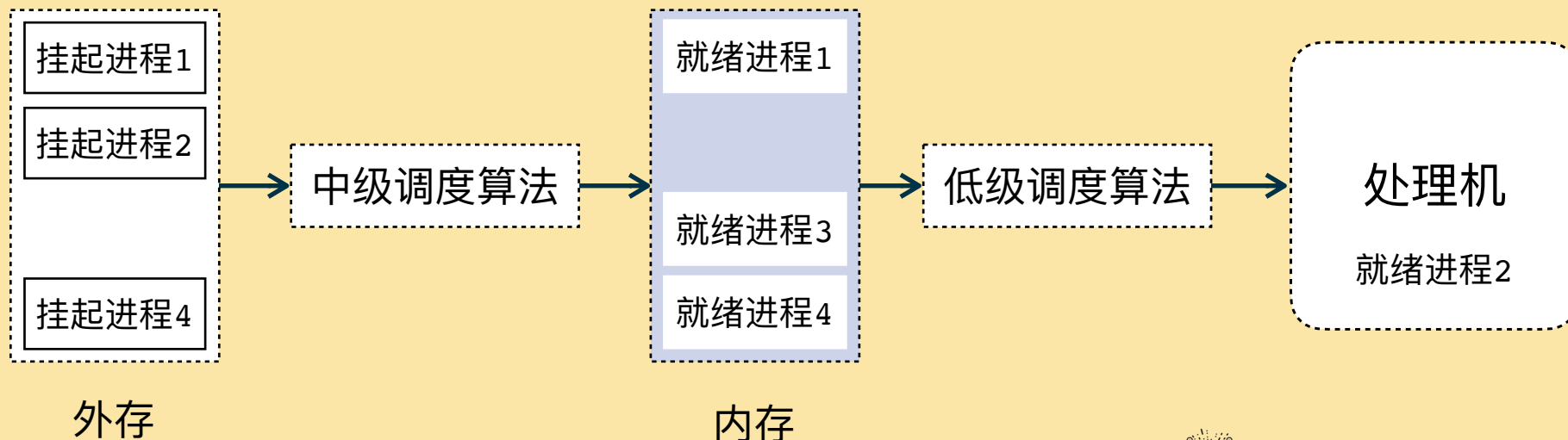
此时进程的状态称为**挂起态**（也叫就绪驻外存状态）

当它们已具备运行条件且内存稍有空闲时

由中级调度来决定把外存上的那些已具备运行条件的就绪进程再次调入内存

并修改它们的状态为就绪状态，挂在就绪队列上等待

中级调度实际上就是存储器管理中的对换功能（后续会介绍）





层次

目标

算法

多处理机调度

低级调度(进程调度)的任务

1. 保存CPU现场信息

进程调度时首先需要保存当前进程的CPU现场信息，如程序计数器、通用寄存器中的内容等

后续关注公众号[研大]
永久微信1951657888

2. 按照某种算法选取进程

调度程序按照算法从就绪队列中选取一个进程，将其状态改为运行状态，并准备把CPU分配给它

3. 将CPU分配给进程

由分派程序把CPU分配给该进程，此时需要选中进程的PCB内有关CPU的现场信息装入CPU相应的各个寄存器中，并把CPU的控制权交给该进程，以使其能从上次的断点处恢复运行





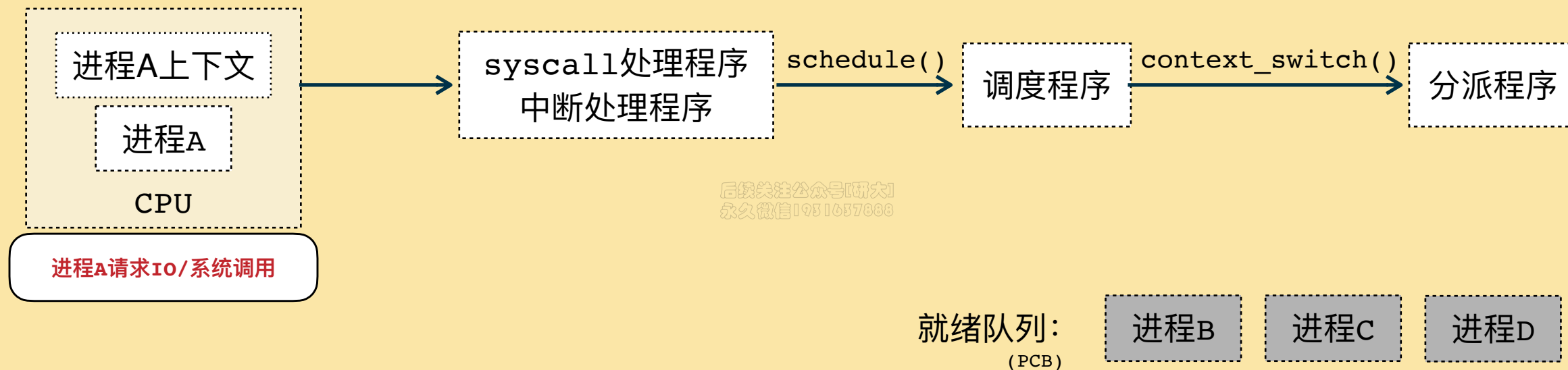
层次

目标

算法

多处理机调度

进程调度机制 (Linux为例)





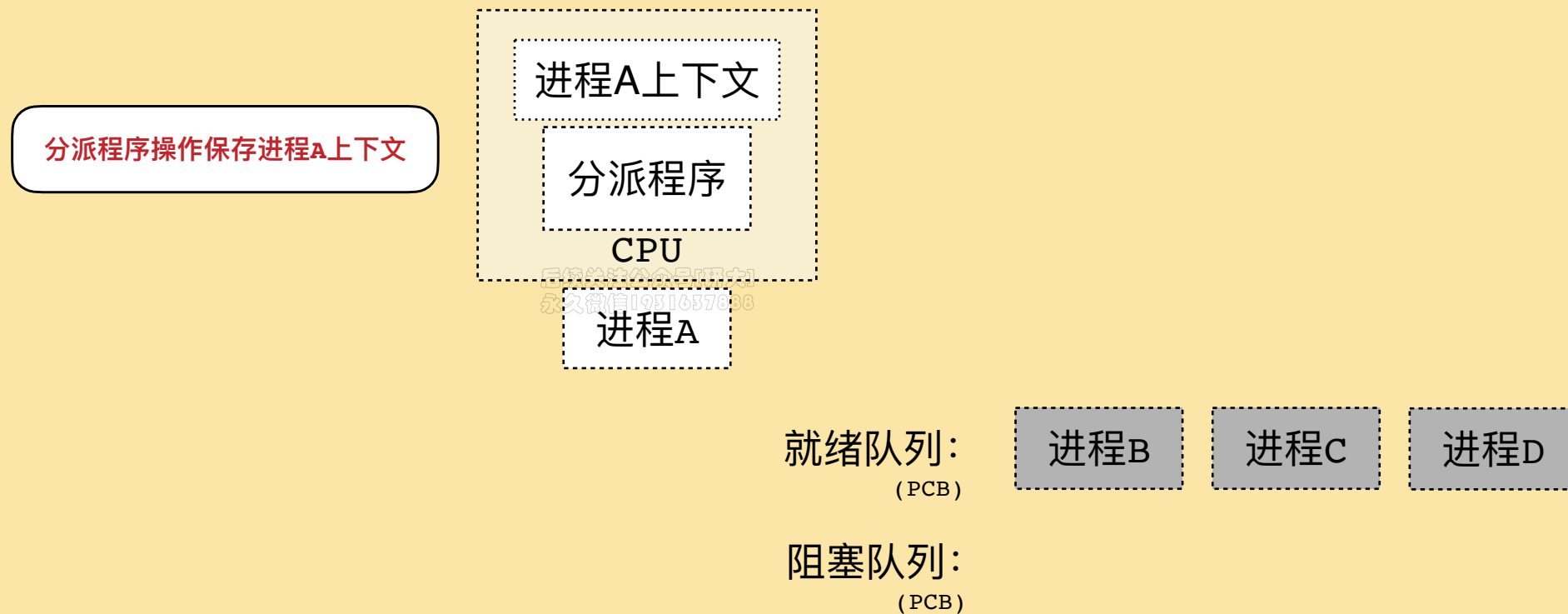
层次

目标

算法

多处理机调度

进程调度机制(Linux为例)





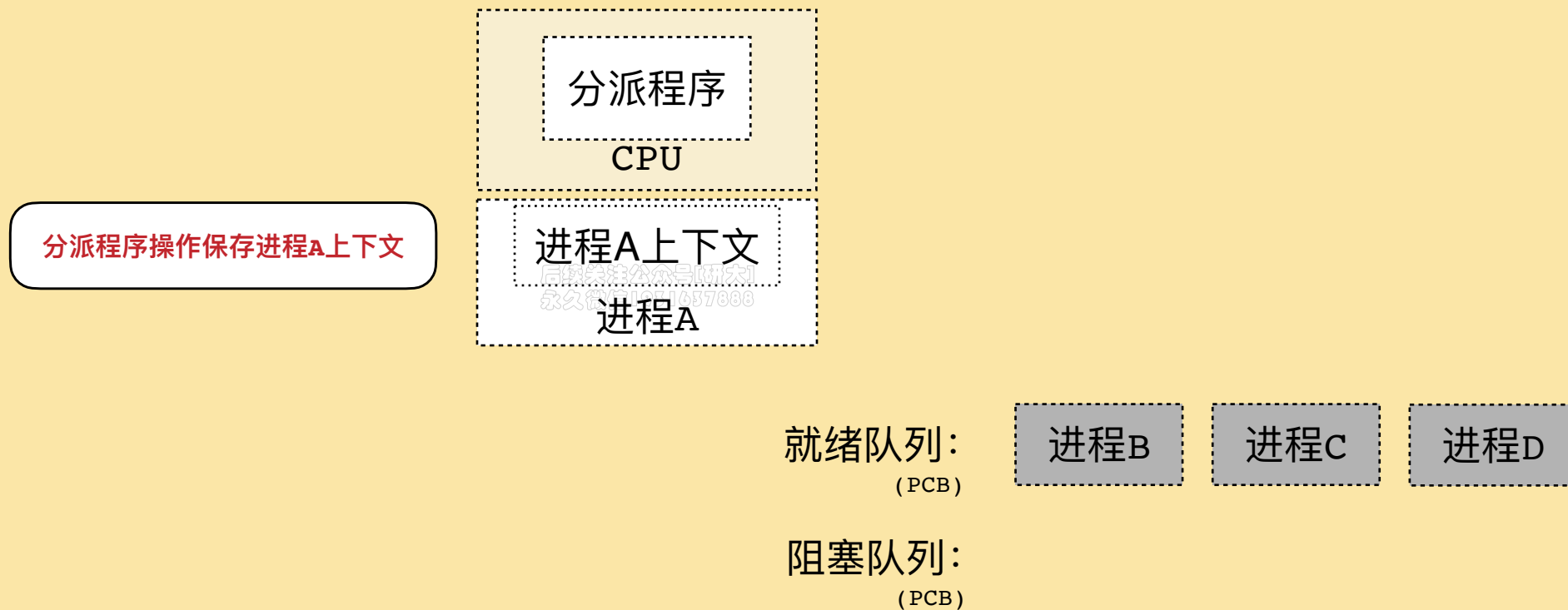
层次

目标

算法

多处理机调度

进程调度机制 (Linux为例)





层次

目标

算法

多处理机调度

进程调度机制(Linux为例)



后续关注公众号[研大]
永久微信1951657888

随后恢复就绪队列中将上机进程上下文

就绪队列:
(PCB)

进程B

进程C

进程D

阻塞队列:
(PCB)

进程A上下文

进程A





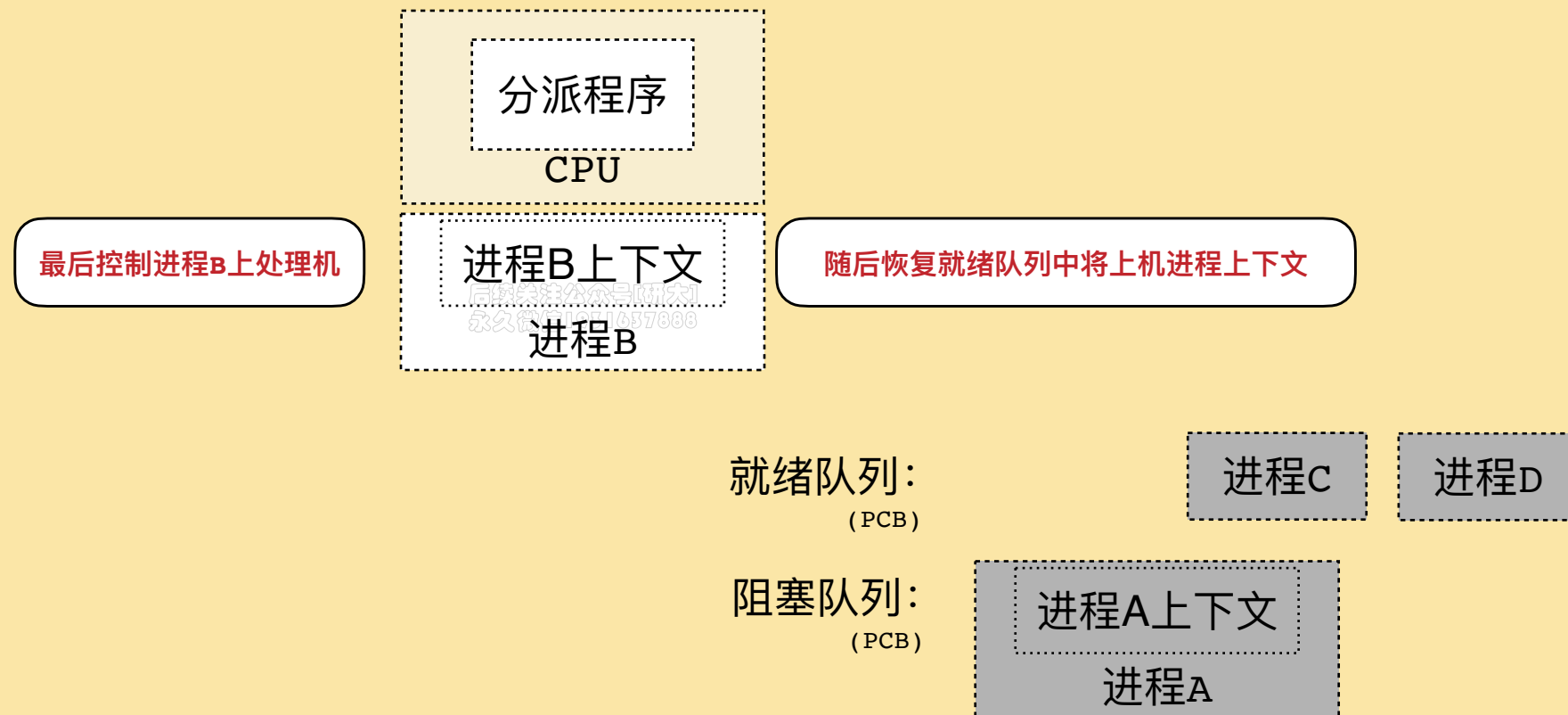
层次

目标

算法

多处理机调度

进程调度机制 (Linux为例)





层次

目标

算法

多处理机调度

进程调度机制 (Linux为例)

最后控制进程B上处理机

进程B上下文

进程B

CPU

就绪队列:
(PCB)

进程C

进程D

阻塞队列:
(PCB)

进程A上下文

进程A





层次

目标

算法

多处理机调度

进程调度方式

非抢占调度方式：一旦把处理机分配给某进程，就会**一直让它运行下去**，而决不会因为时钟中断或其他原因去抢占该进程的处理机，直至该进程完成或发生某事件而被阻塞，才会把分配给该进程的处理机分配给其他进程

非抢占方式下可能引起进程调度的因素：

- 1.正在执行进程执行完毕或无法继续执行；
- 2.正在执行的进程因提出IO请求而暂停执行；
- 3.进程通信或同步过程中执行了某种原语操作(如block原语)

抢占调度方式：允许调度程序根据某种原则去**暂停正在执行的进程**，并将**已分配给该进程的处理机重新分配**给另一进程

在现代os中广泛采用抢占调度方式

对于批处理机系统，抢占调度方式可以防止一个长进程长时间占用处理机

分时系统中，只有抢占调度方式才可能实现人机交互

实时系统中，抢占调度方式能满足实时任务的需求

抢占不是随意的抢占，必须遵循一些原则，主要有：

- 1.优先级高可抢占处理机；
- 2.短进程优先原则；
- 3.时间片原则，各进程按时间片运行，正在执行的进程的一个时间片用完后，停止该进程执行并重新调度





层次

目标

算法

多处理机调度

进程调度时机

主动放弃

1. 运行完毕，正常终止

2. 运行过程中发生异常而终止

3. 发生事件，请求阻塞

被动放弃

1. 进程时间片用完

2. 有高优先级进程加入就绪队列

3. 有更加紧急事情需要处理

后续关注公众号「研大」
永久微信1931637888





层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

CPU利用率

$\text{CPU有效工作时间} / (\text{CPU有效工作时间} + \text{CPU空闲等待时间})$

系统吞吐量

单位时间内CPU完成作业的数量

周转时间

作业完成时间 - 作业提交时间

平均周转时间

作业周转时间之和 / 作业数量

带权周转时间

作业周转时间 / 作业实际运行时间

平均带权周转时间

带权周转时间之和 / 作业数量

等待时间

进程等待CPU时间之和

响应时间

从用户提交请求到系统首次产生响应所用时间

后续关注公众号「研大」
永久微信1951657888





层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

后续关注公众号[研大]
永久微信1951657886

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用非抢占式短进程优先调度算法,求这5进程的平均周转时间

若采用抢占式短进程优先调度算法,求这5进程的平均周转时间





层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

平均周转时间

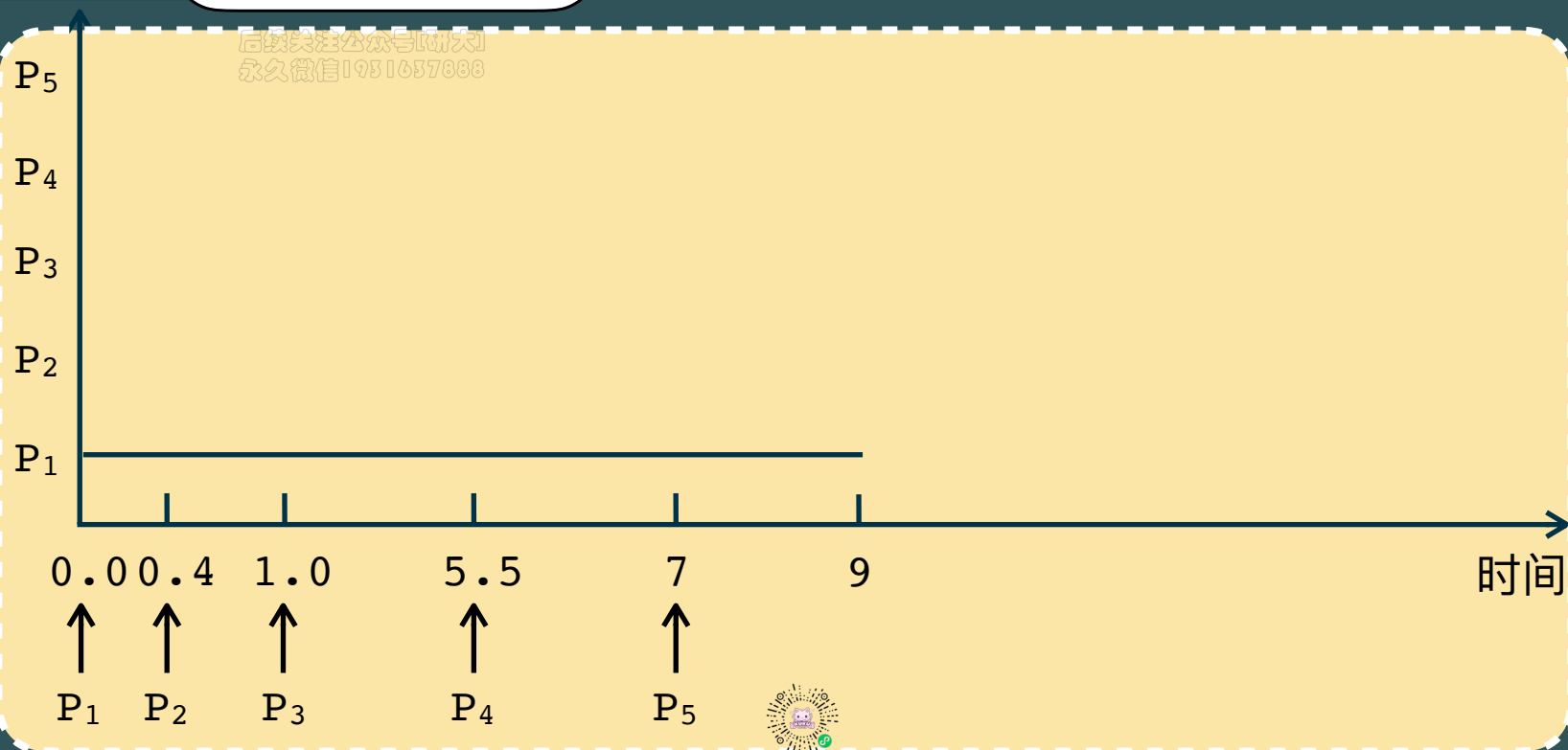
作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用非抢占式短进程优先调度算法
求这5进程的平均周转时间

一开始只有P₁
运行P₁
并且因非抢占式
P₁运行结束之前不会调度





层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

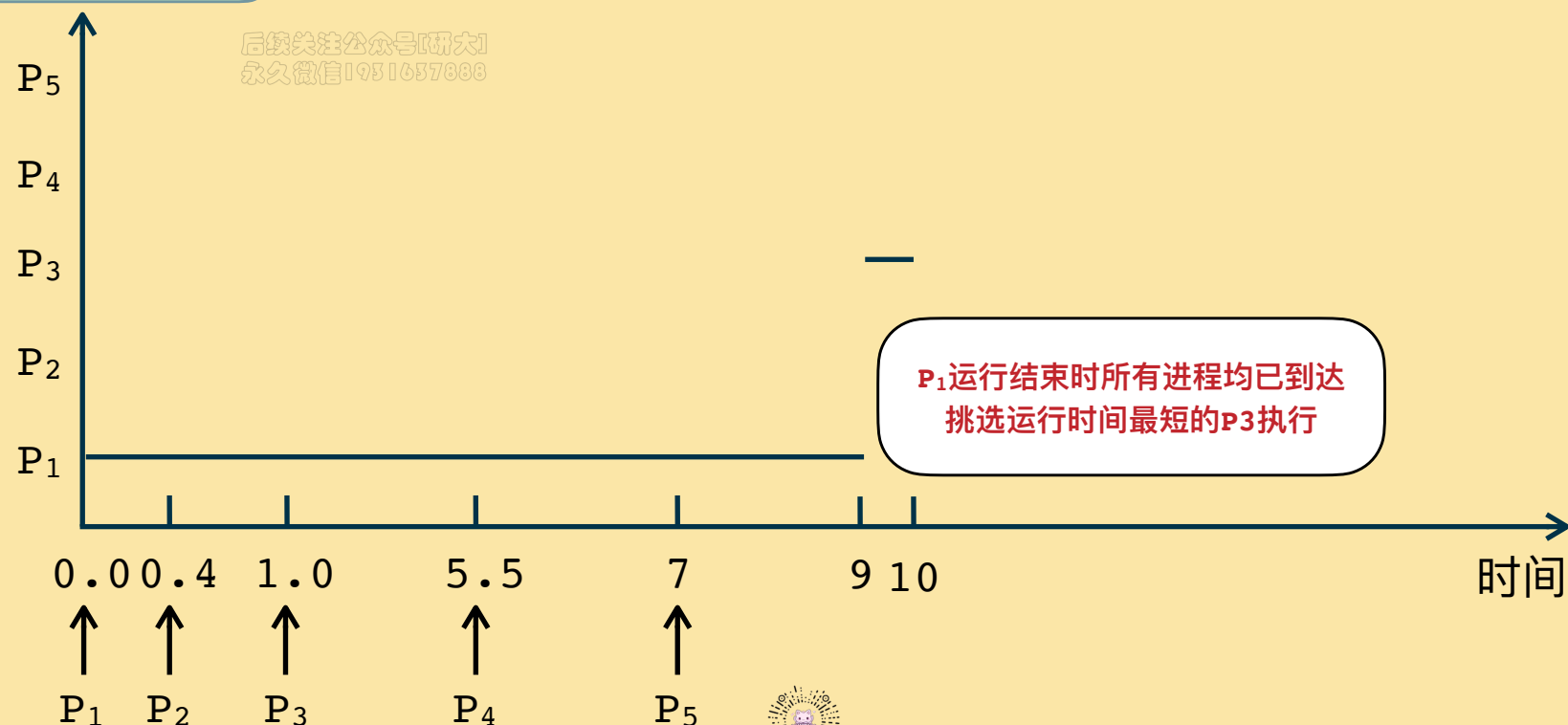
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用非抢占式短进程优先调度算法
求这5进程的平均周转时间





层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

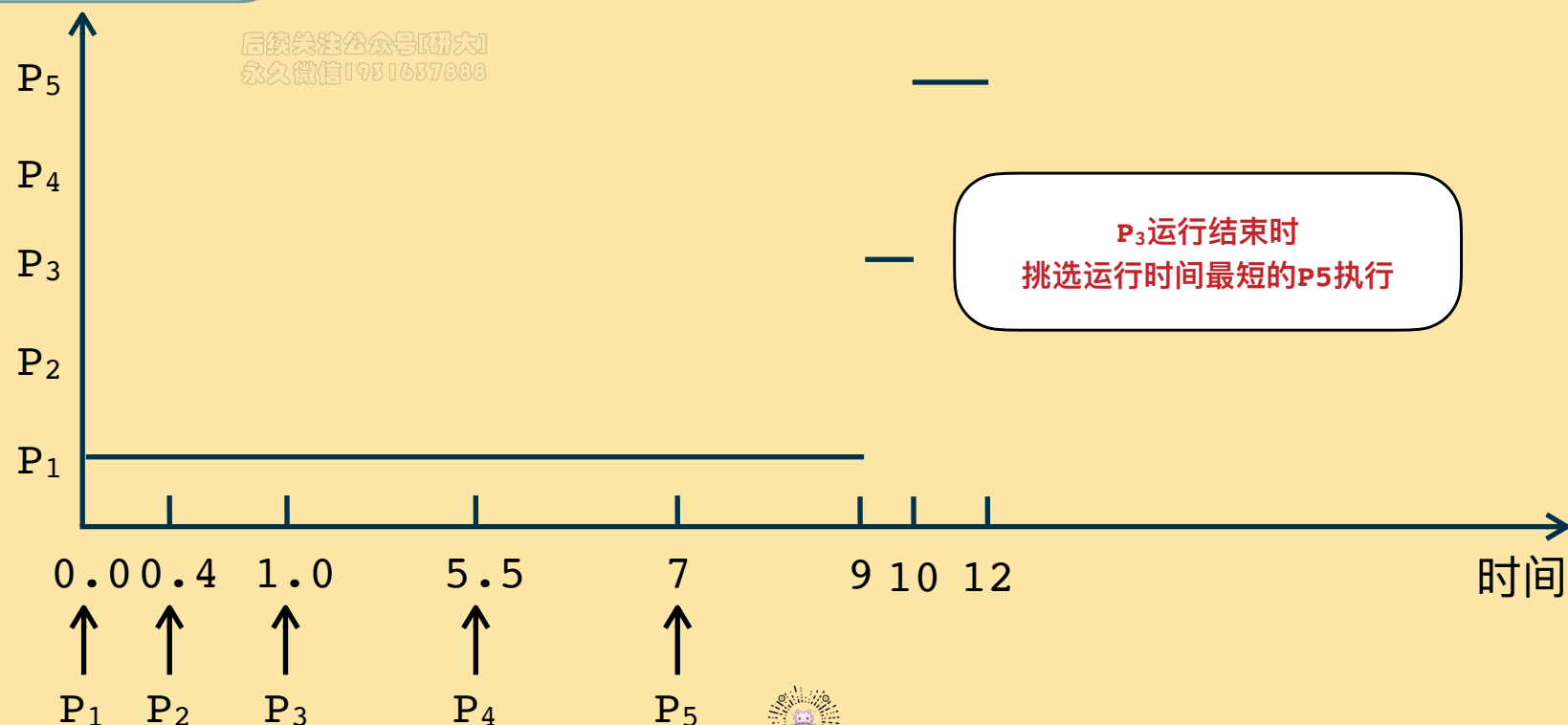
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用非抢占式短进程优先调度算法
求这5进程的平均周转时间





层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

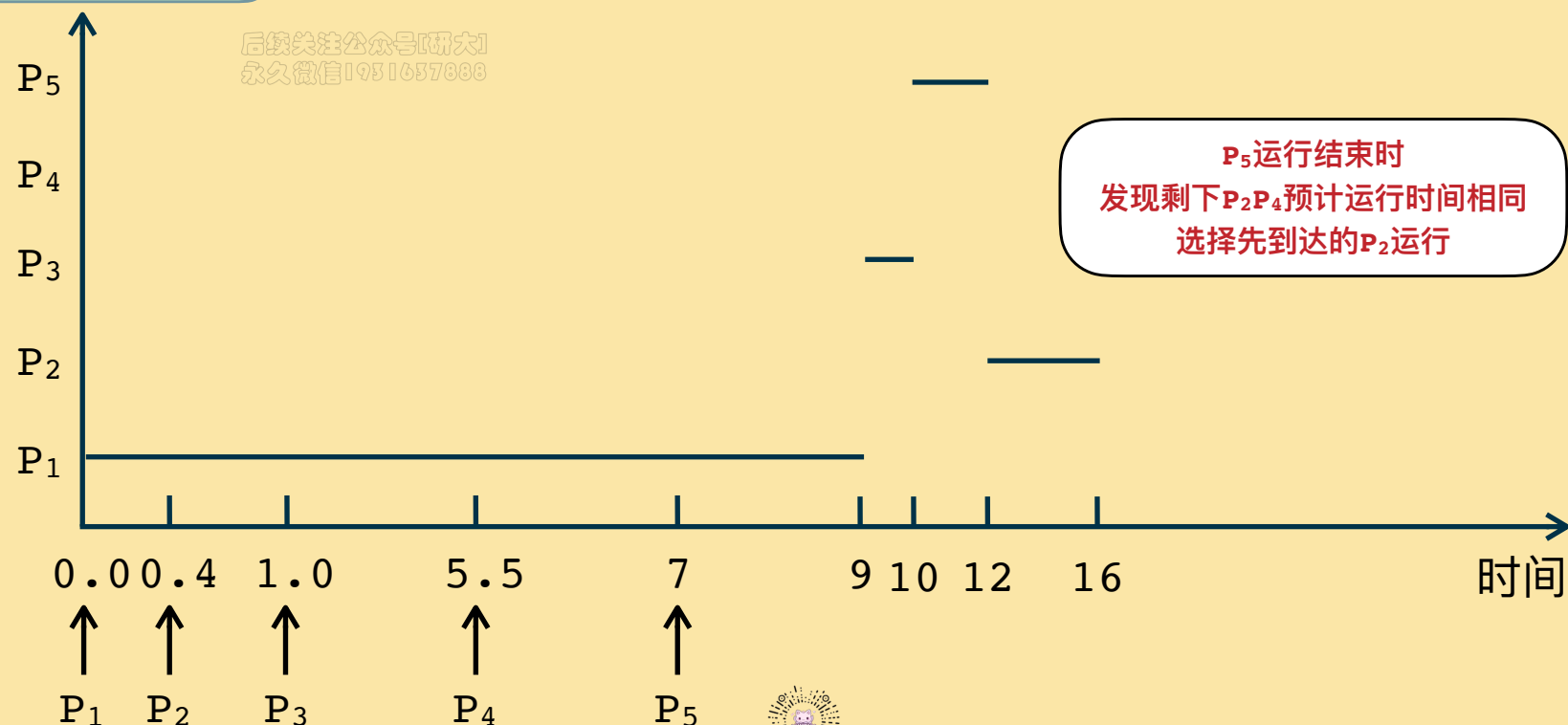
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用非抢占式短进程优先调度算法
求这5进程的平均周转时间





层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

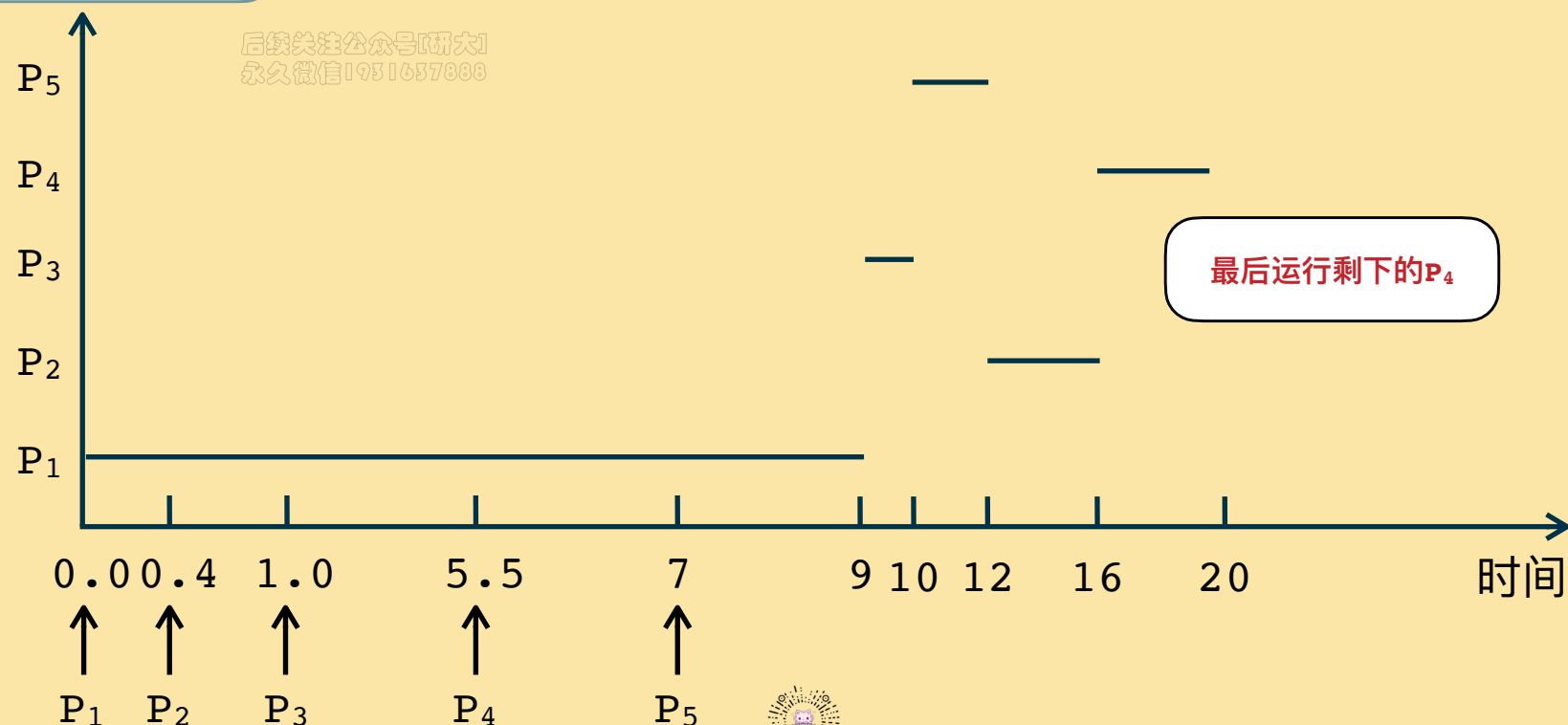
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用非抢占式短进程优先调度算法
求这5进程的平均周转时间





层次

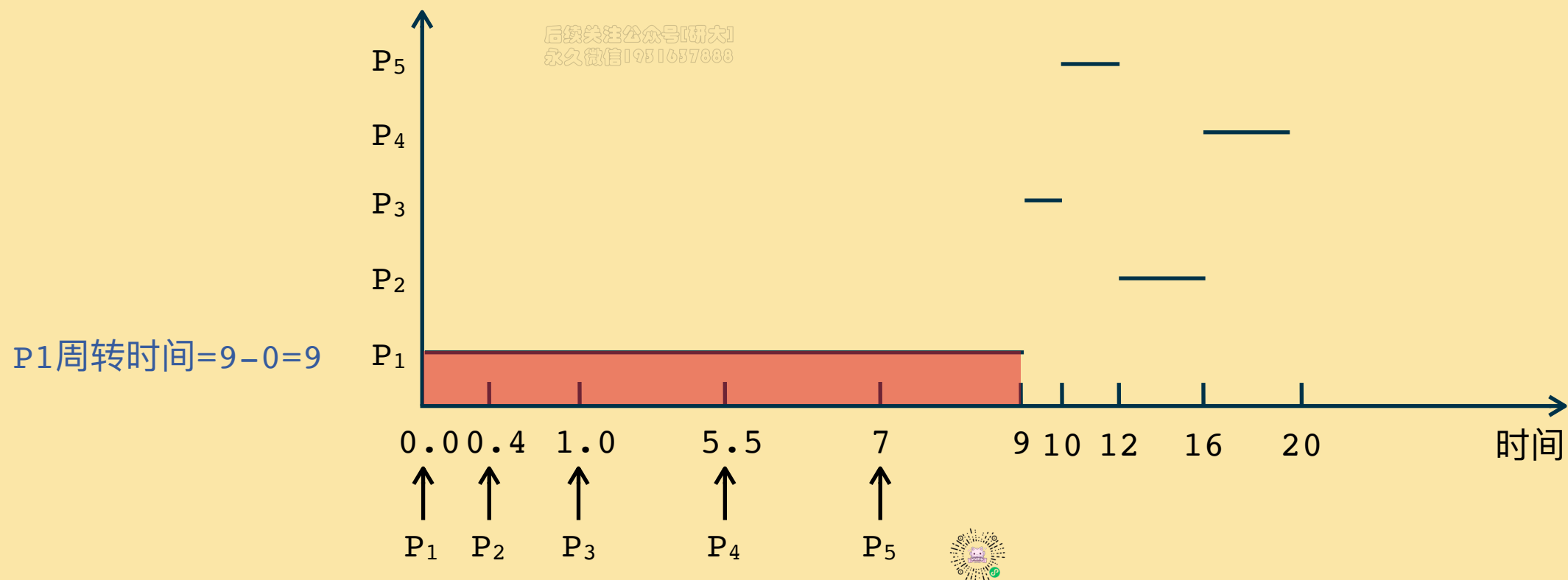
目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准





层次

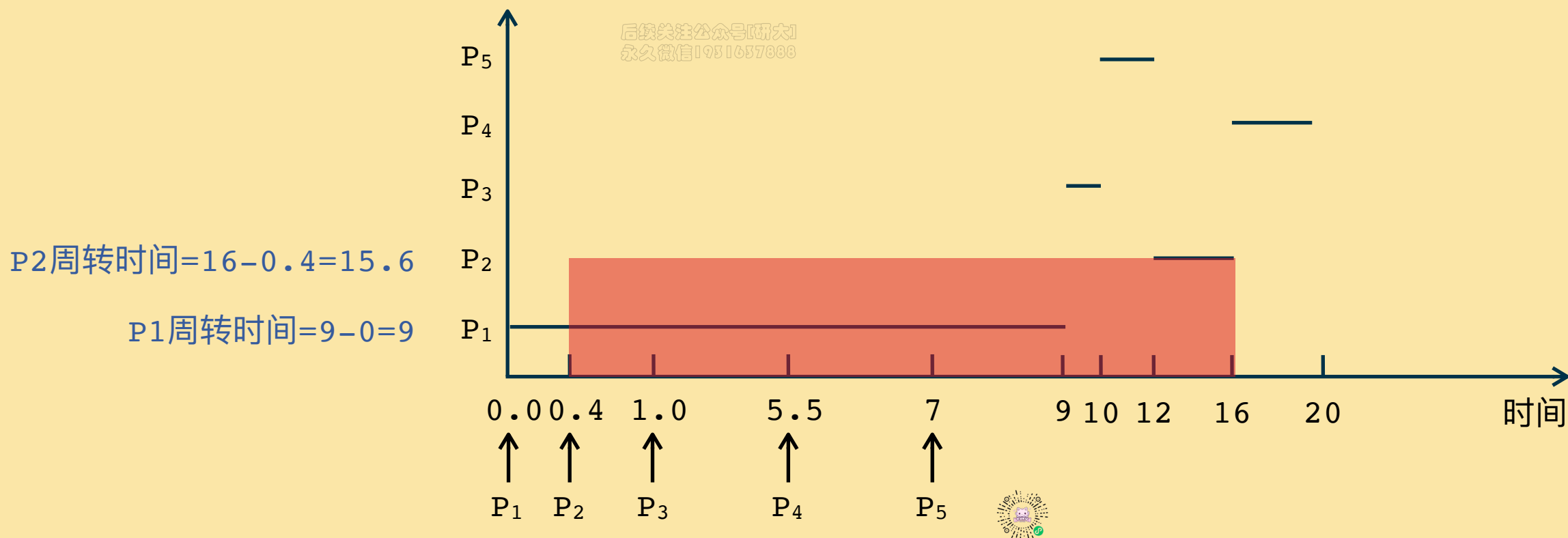
目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准





层次

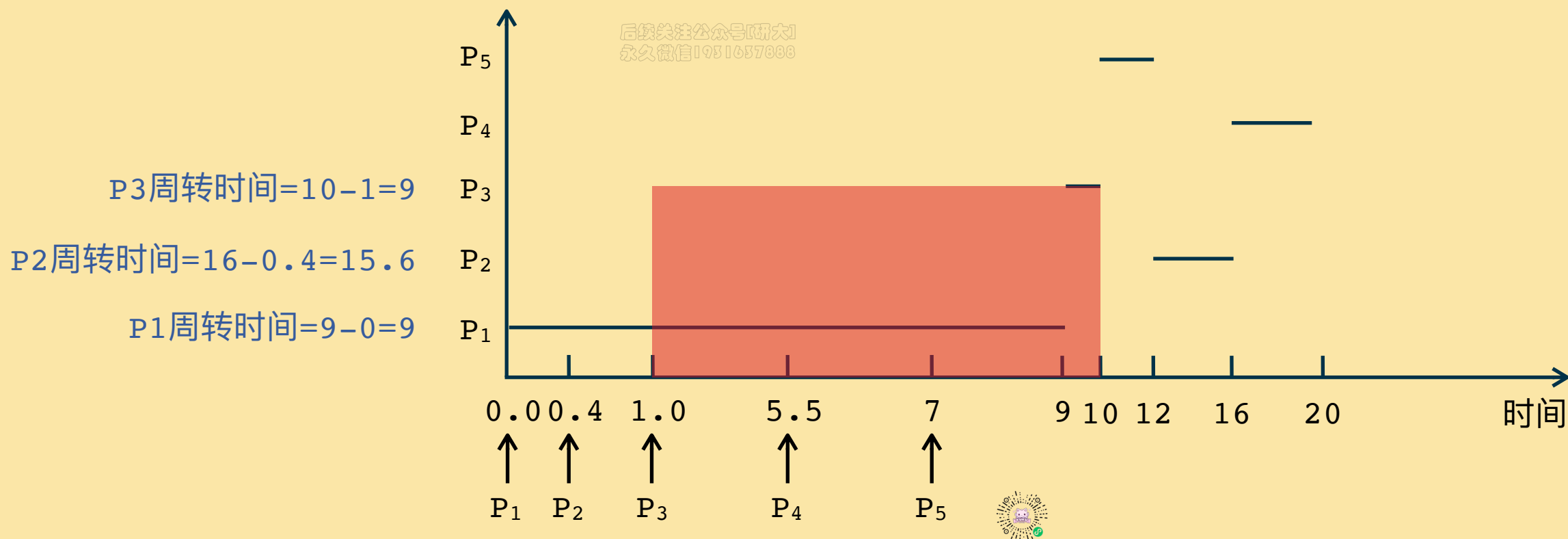
目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准





层次

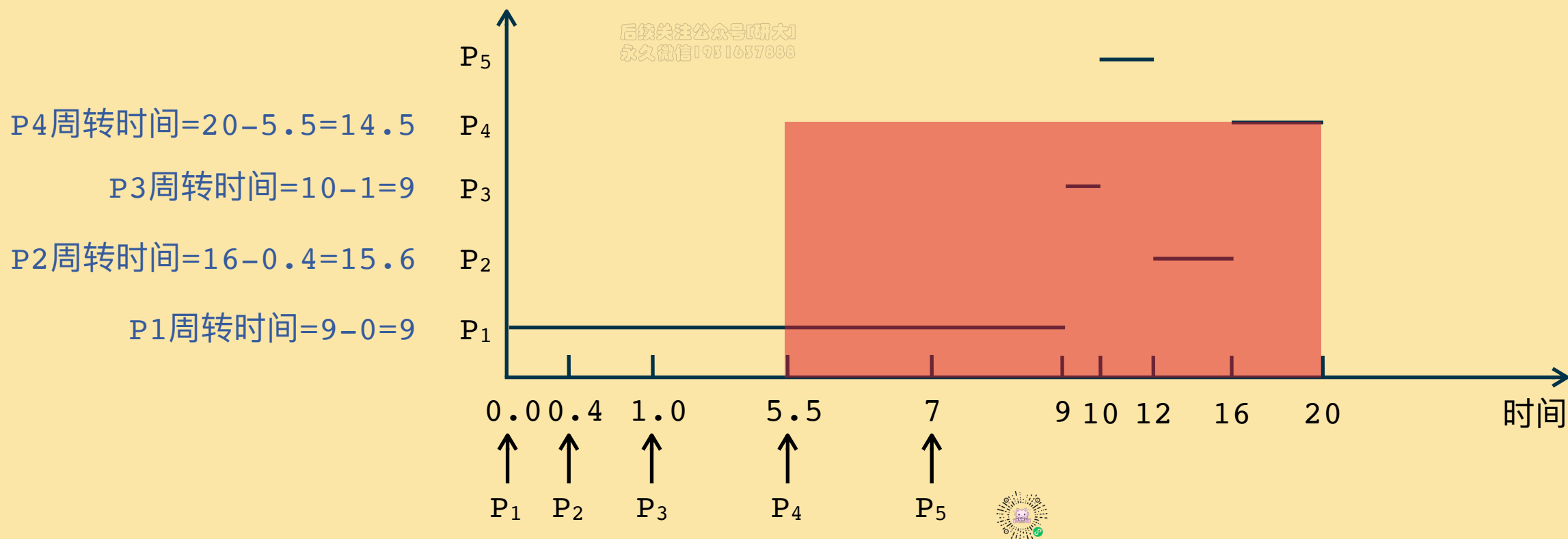
目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准





层次

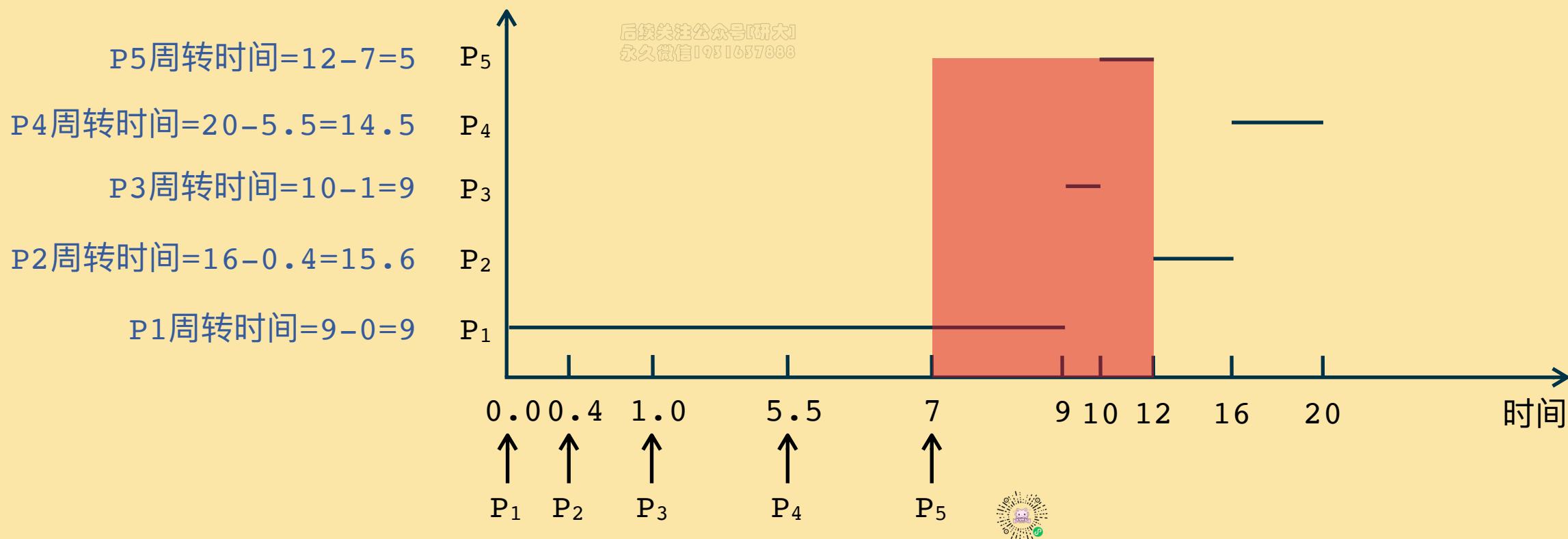
目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准





层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用非抢占式短进程优先调度算法
求这5进程的平均周转时间

$$P_5 \text{ 周转时间} = 12 - 7 = 5$$

$$P_4 \text{ 周转时间} = 20 - 5.5 = 14.5$$

$$P_3 \text{ 周转时间} = 10 - 1 = 9$$

$$P_2 \text{ 周转时间} = 16 - 0.4 = 15.6$$

$$P_1 \text{ 周转时间} = 9 - 0 = 9$$

$$\text{平均周转时间} = (9 + 15.6 + 9 + 14.5 + 5) / 5 = 10.62$$

后续关注公众号「研大」
永久微信1951657888





层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

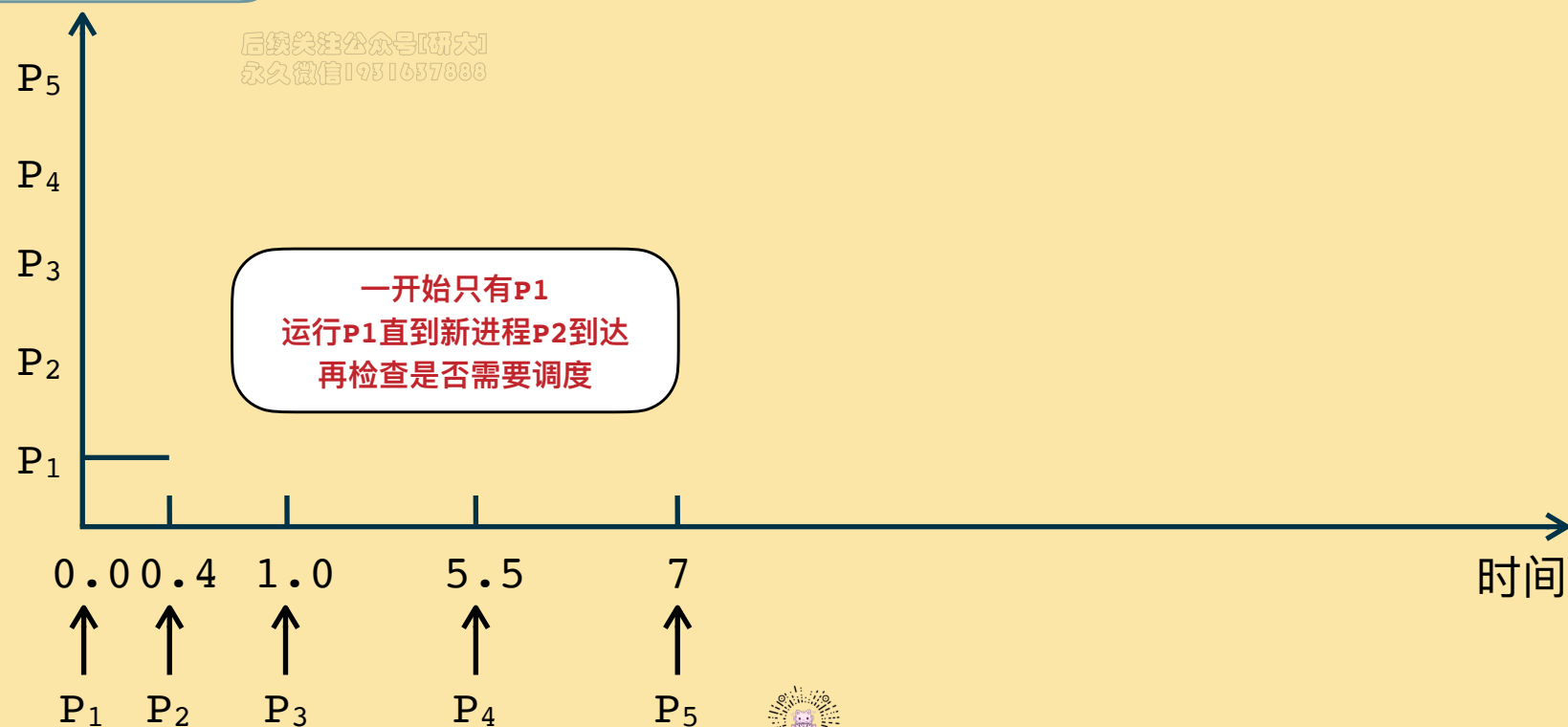
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



t=0.4	剩余运行时间	完成时间
P ₁	8.6	-
P ₂	4	-
P ₃	1	-
P ₄	4	-
P ₅	2	-

处理机调度



- 层次
- 目标
- 算法
- 多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

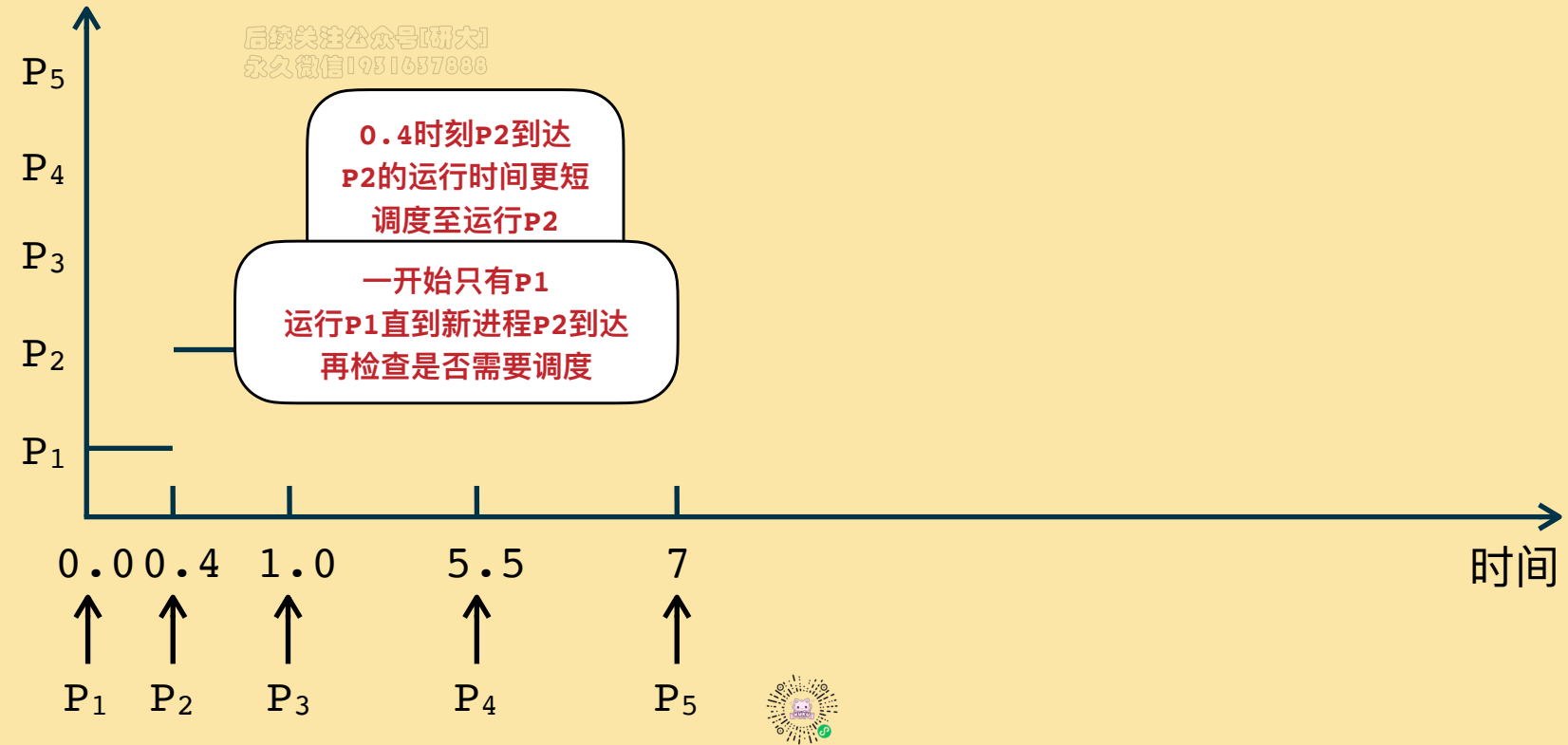
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



处理机调度



层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

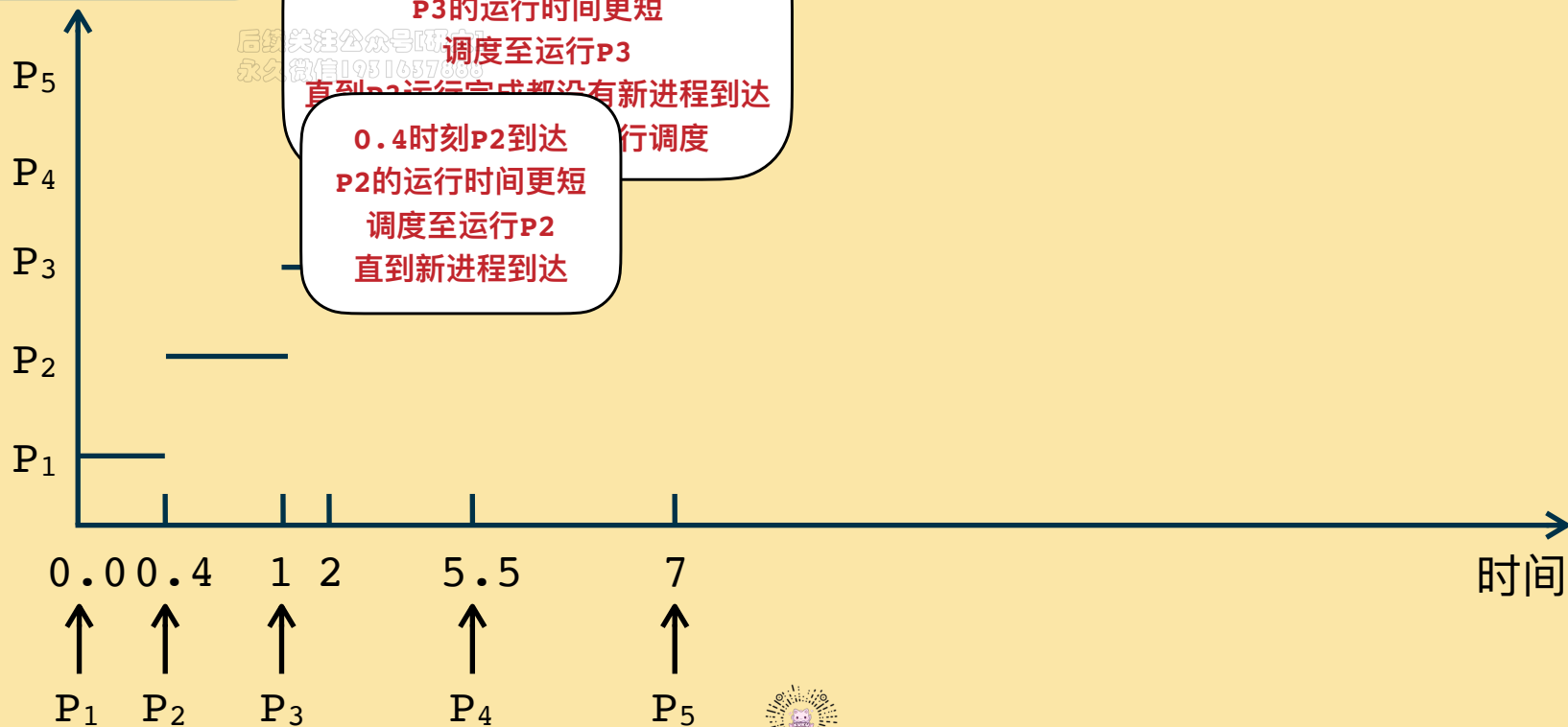
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



处理机调度



BOK操作系统



层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

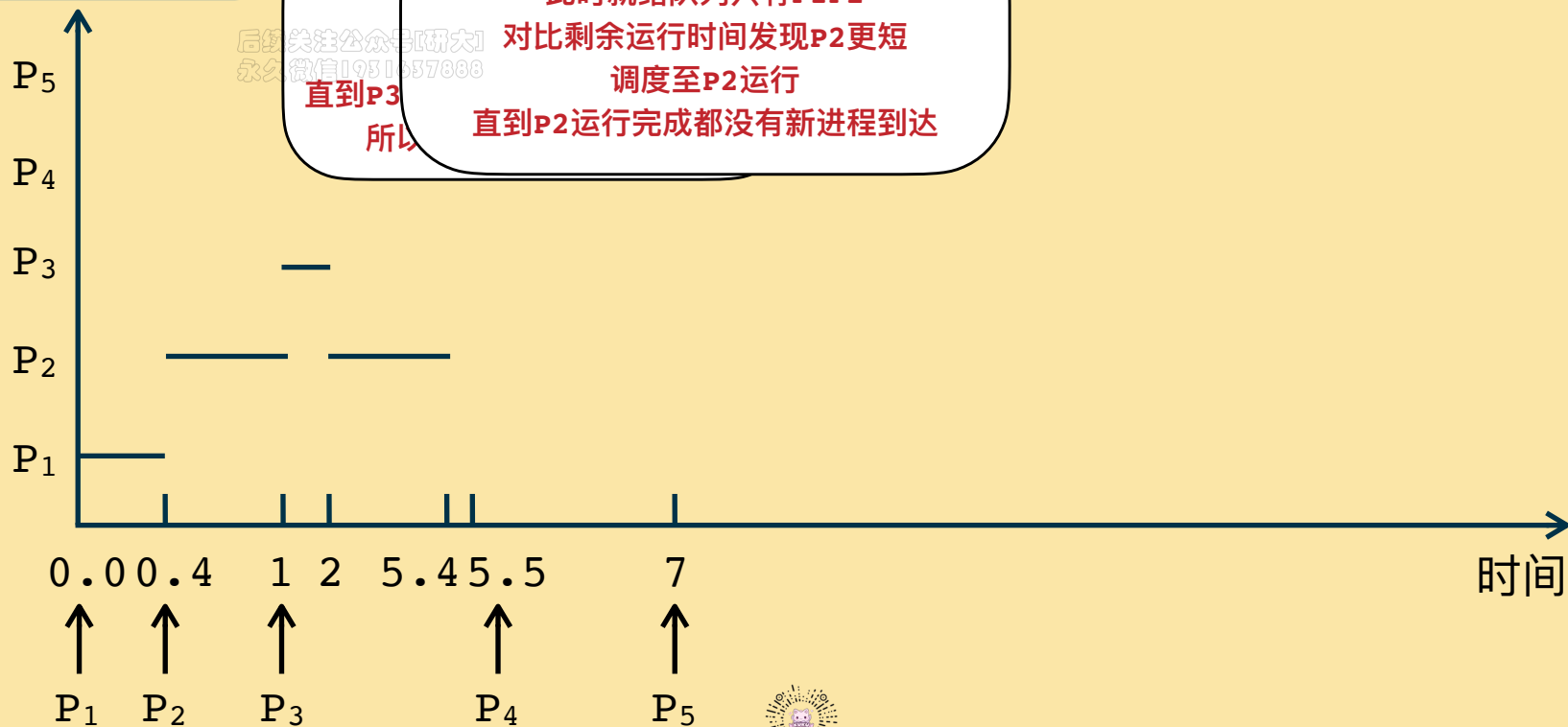
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



处理机调度



BOK操作系统



层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

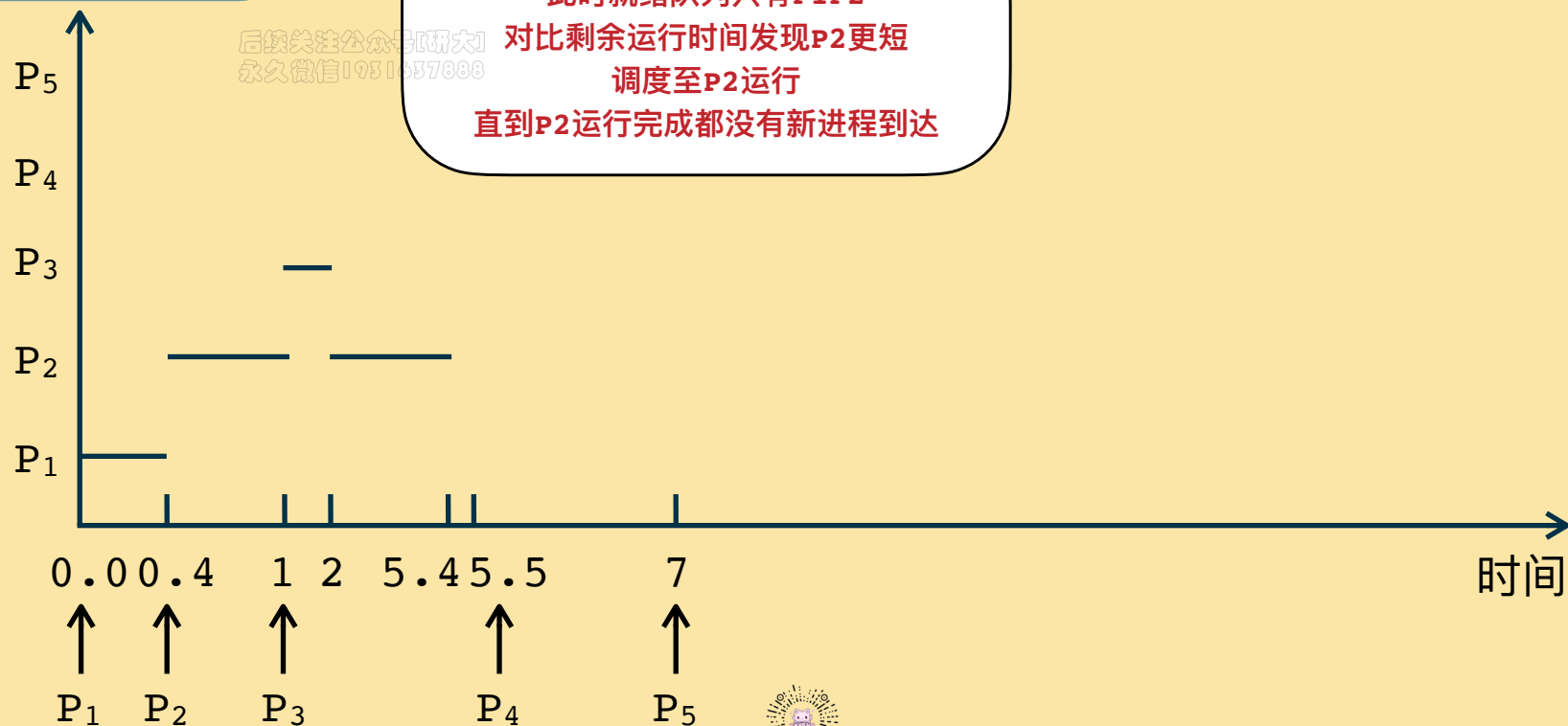
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



处理机调度



BOK操作系统



层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

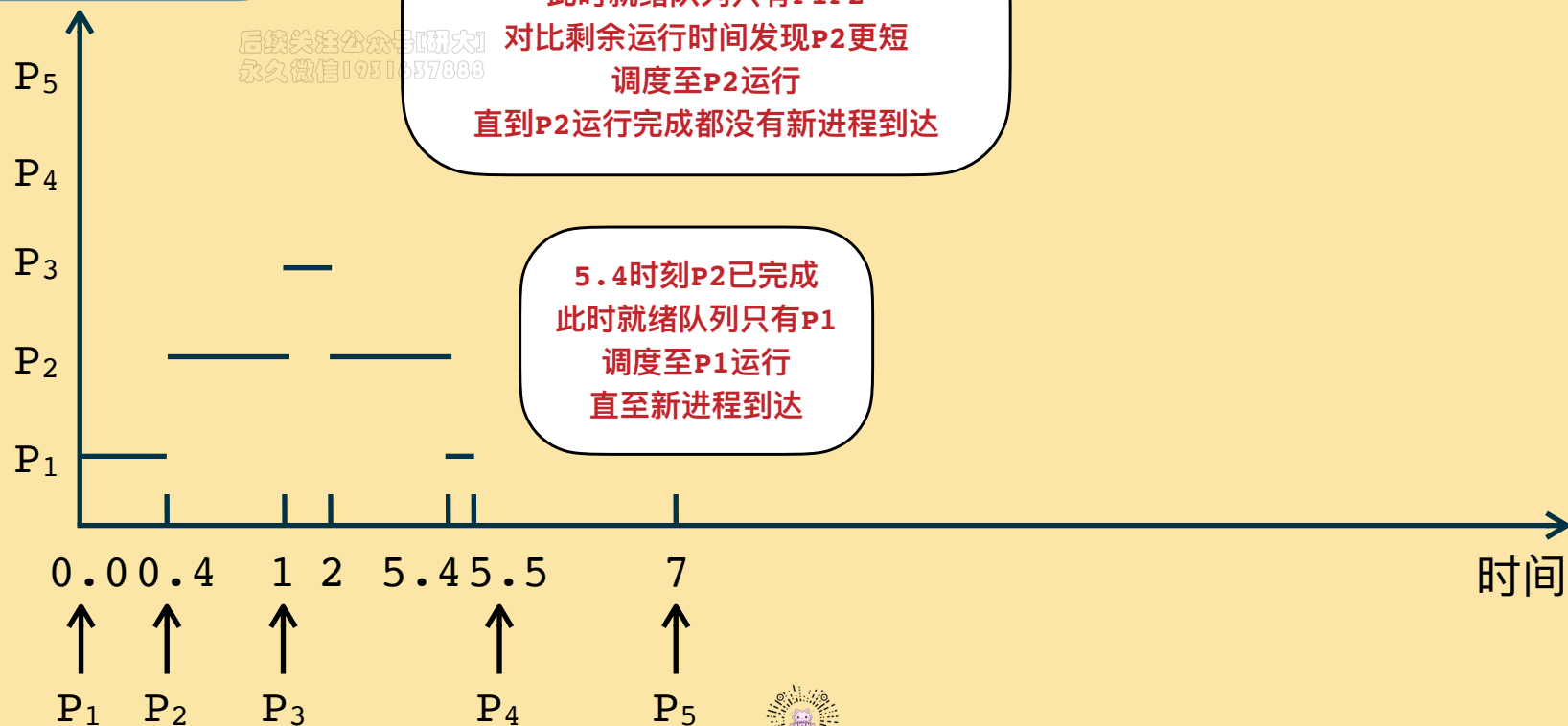
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



t=5.5	剩余运行时间	完成时间
P ₁	8.5	-
P ₂	0	5.4
P ₃	0	2
P ₄	4	-
P ₅	2	-

处理机调度



- 层次
- 目标
- 算法
- 多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

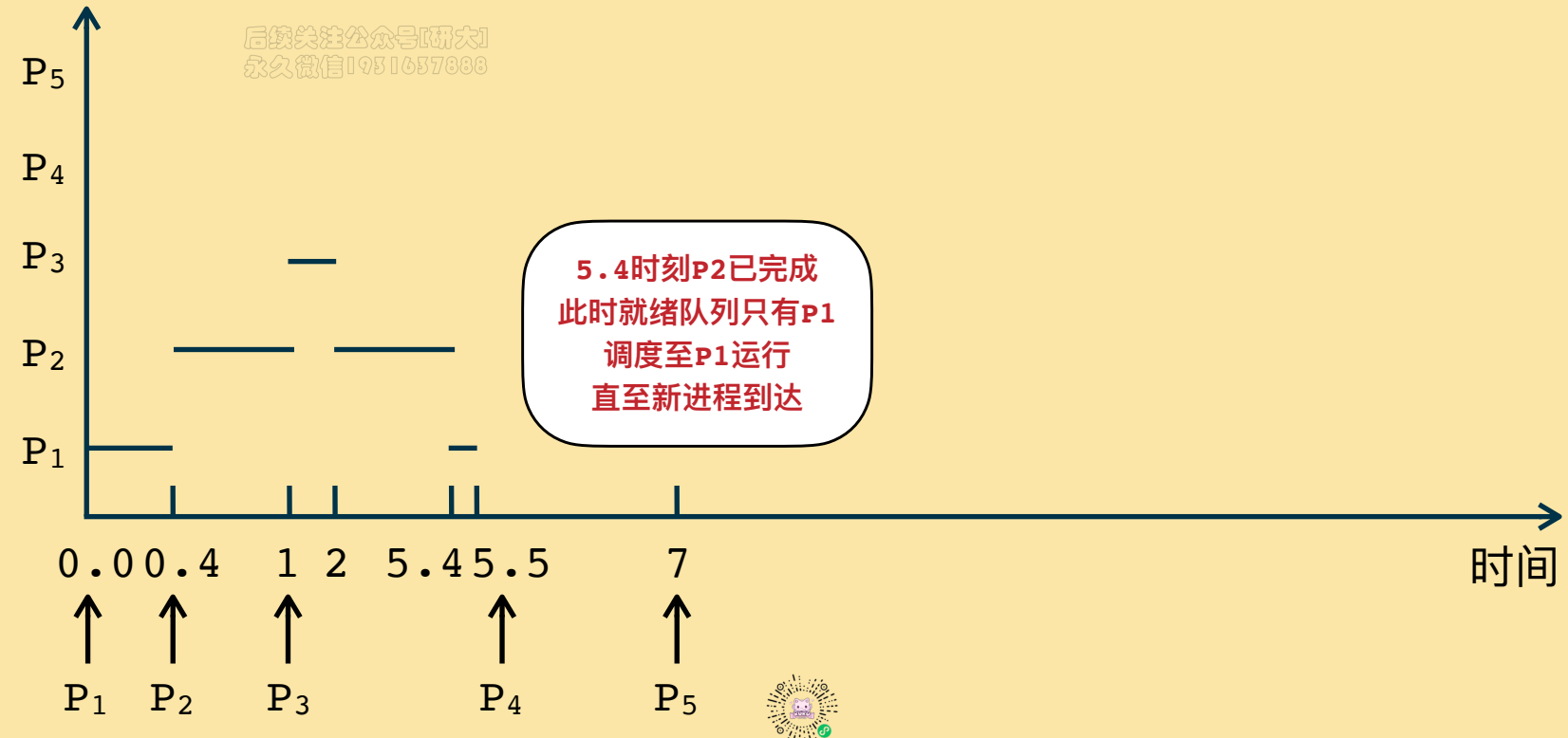
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



处理机调度



BOK操作系统



层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

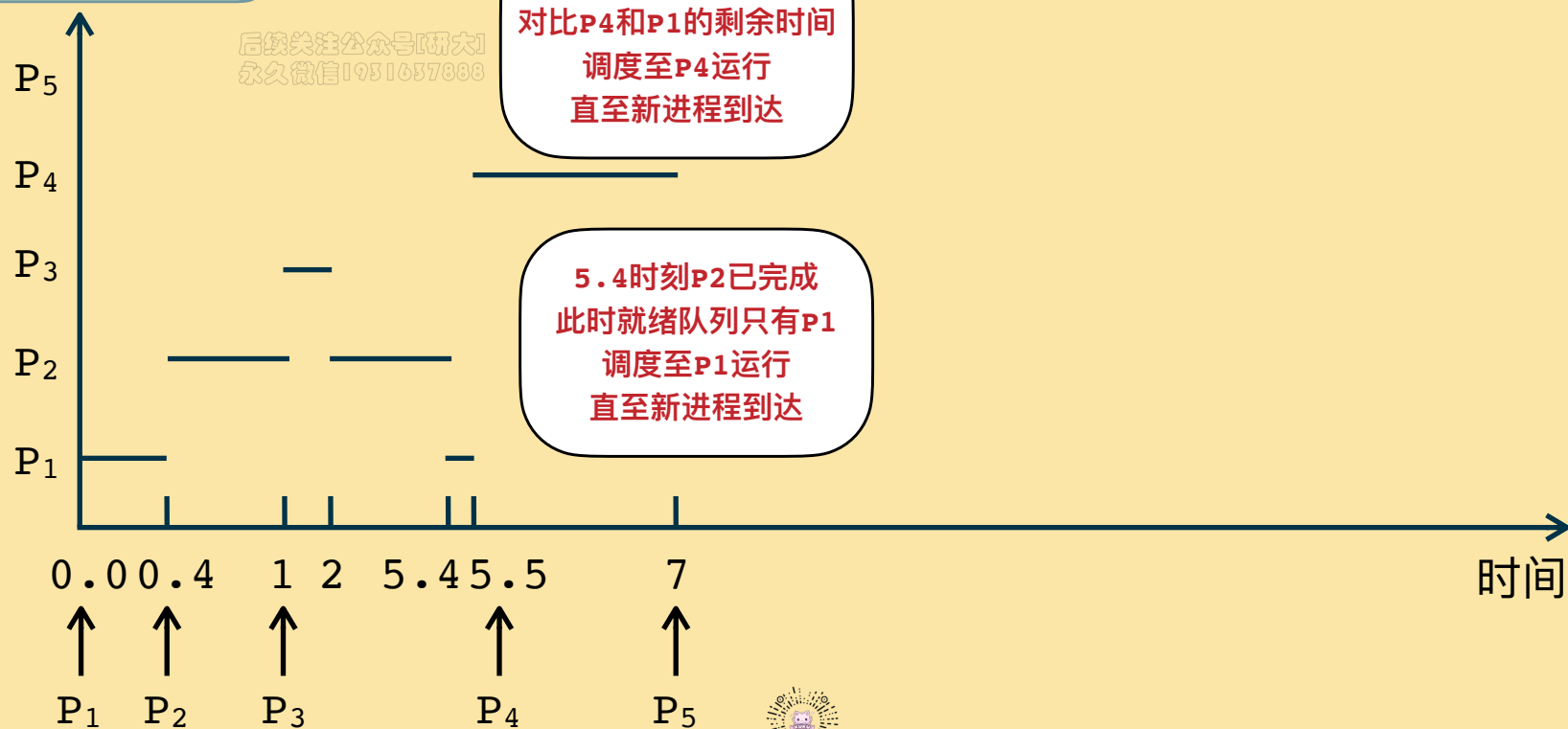
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



处理机调度



BOK操作系统



层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

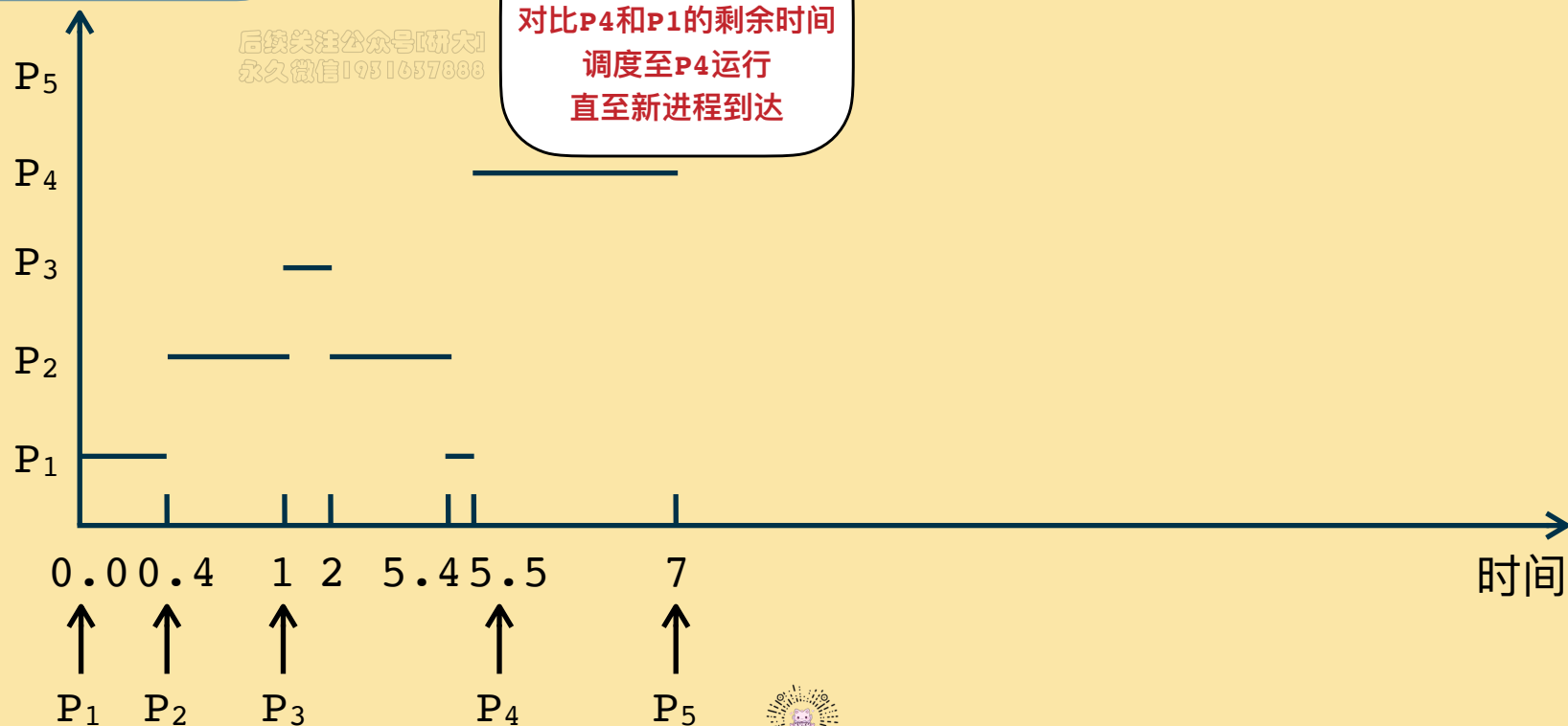
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



处理机调度



BOK操作系统



层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

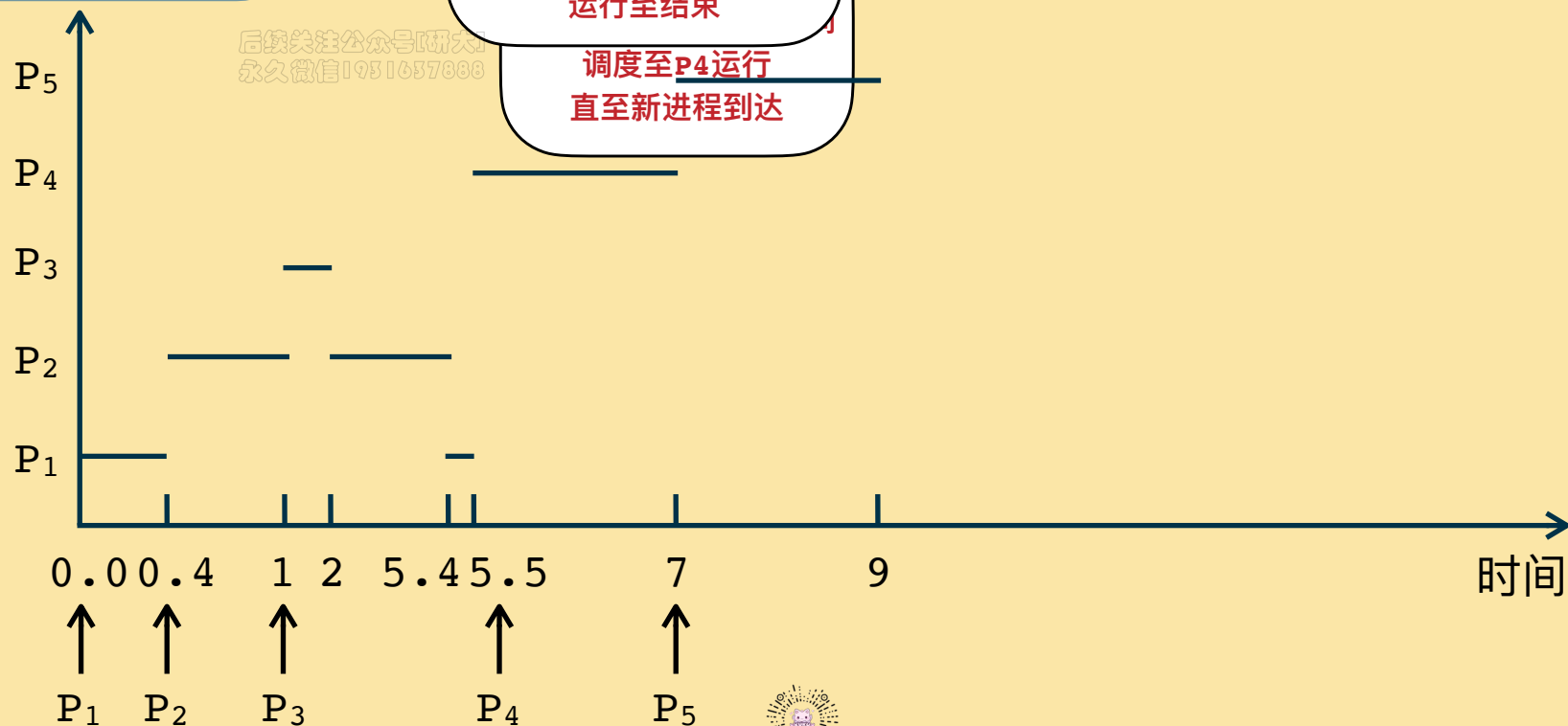
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



处理机调度



BOK操作系统



层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

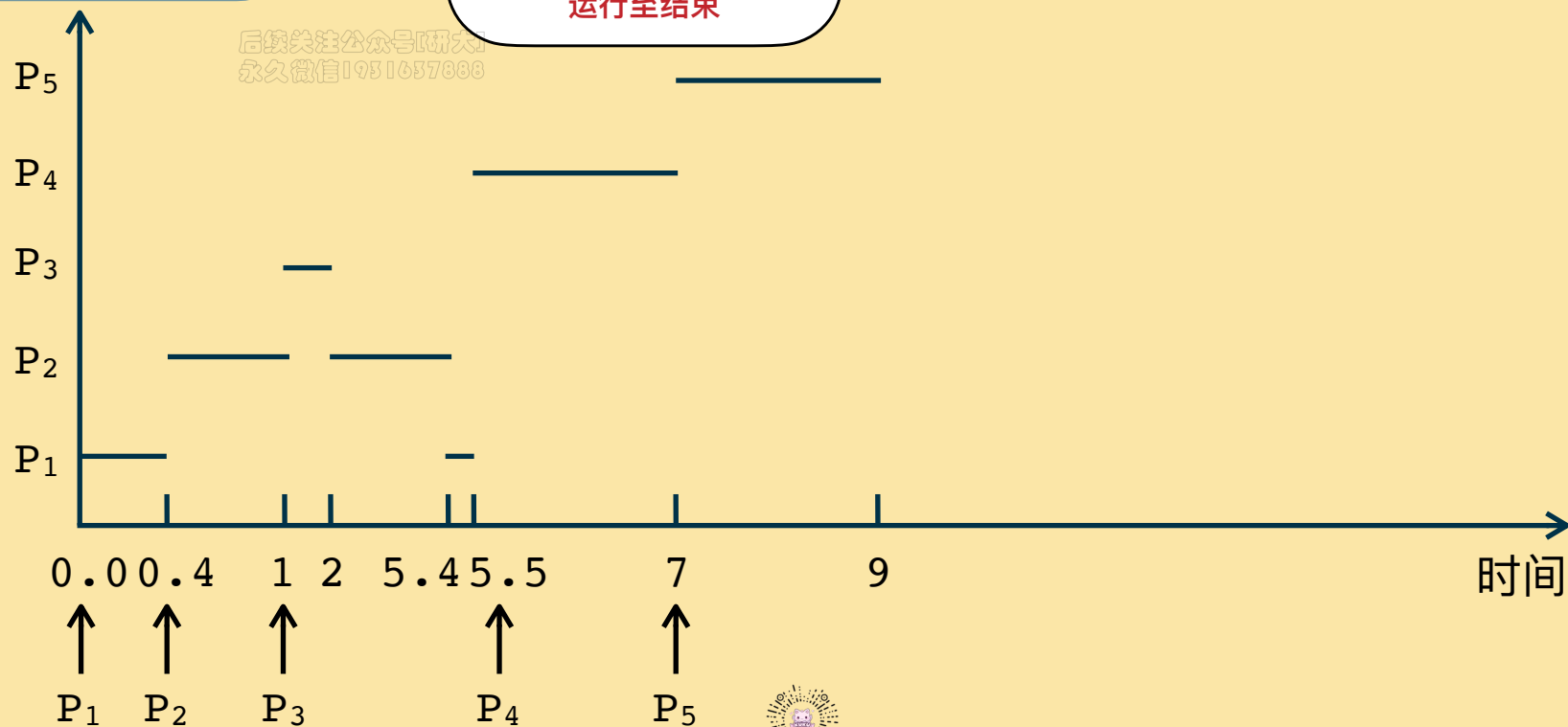
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



处理机调度



BOK操作系统



层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

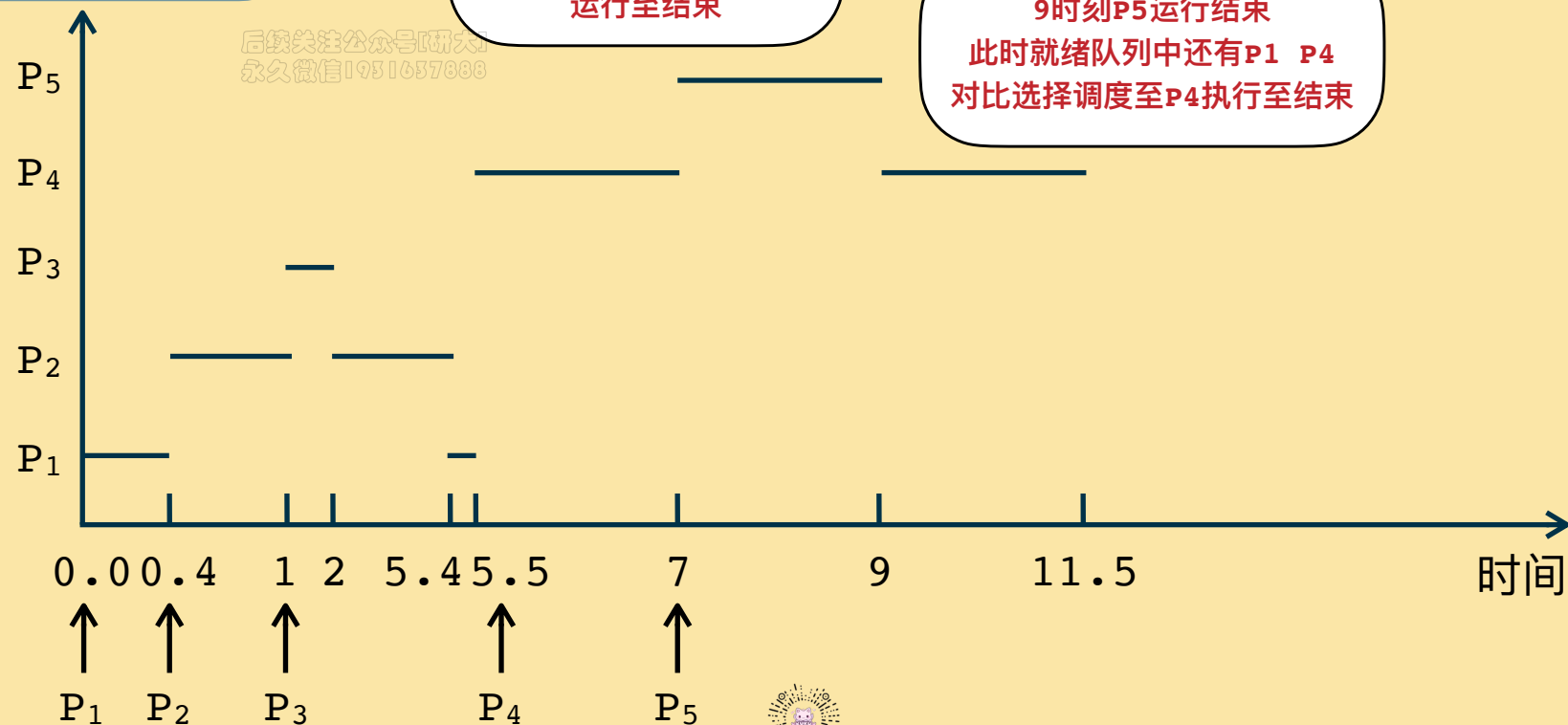
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



处理机调度



BOK操作系统



层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

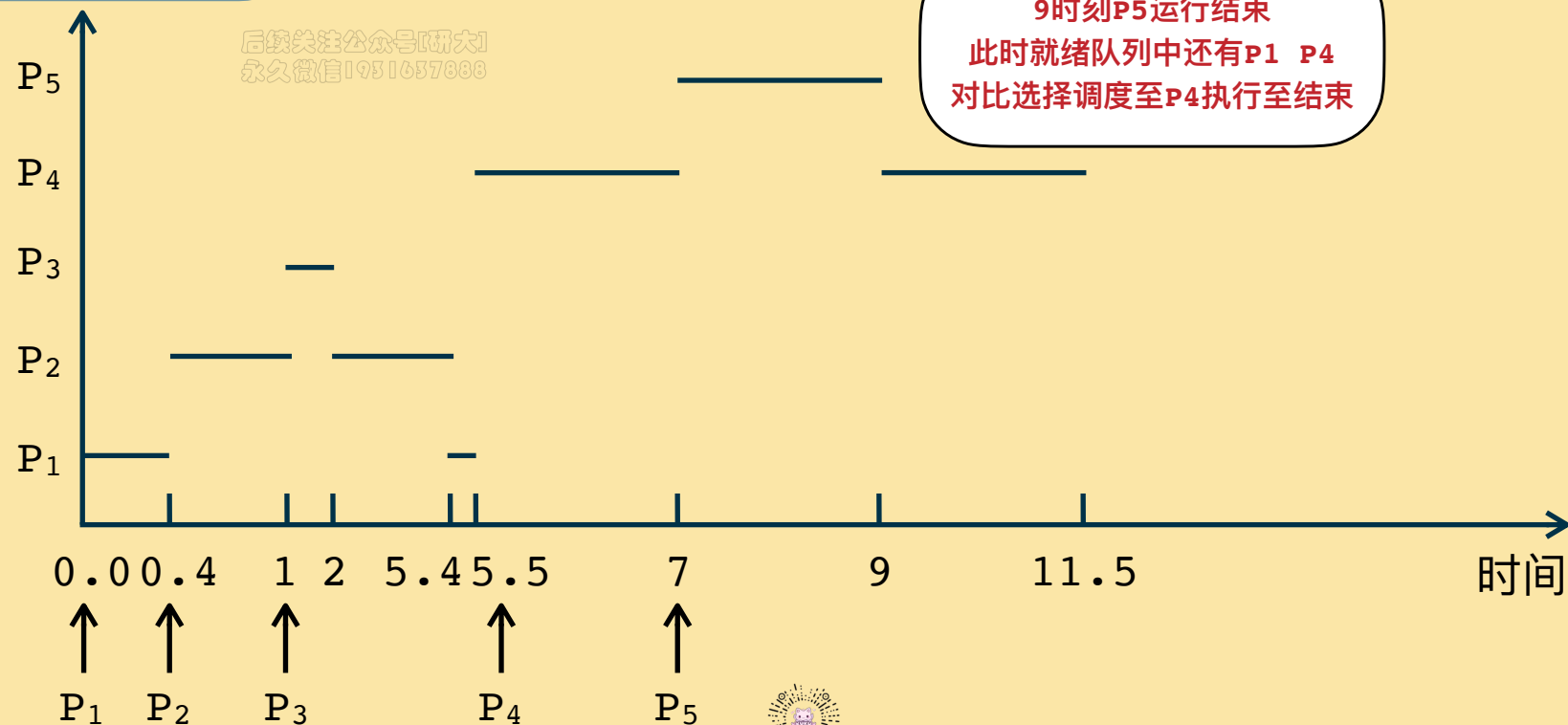
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



处理机调度



BOK操作系统



层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

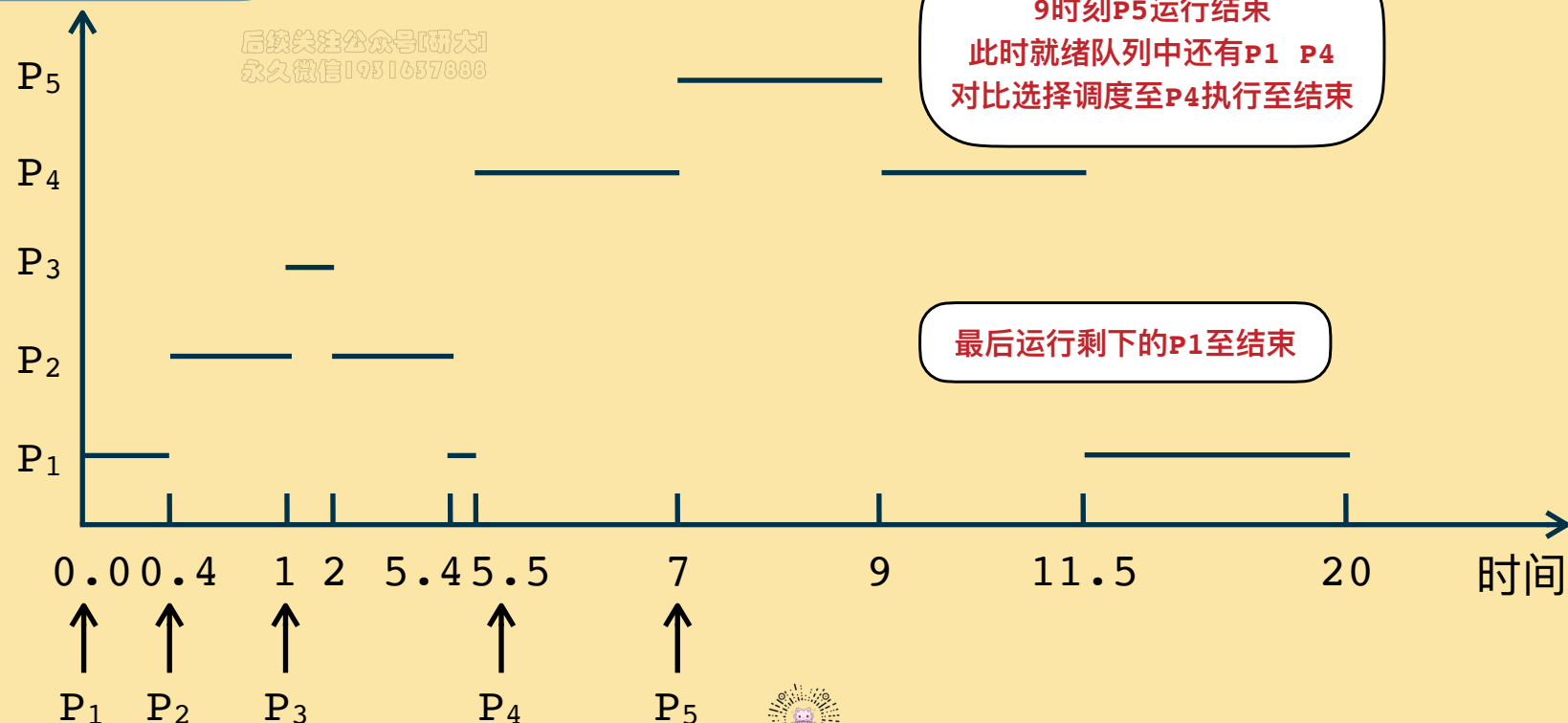
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间



处理机调度



BOK操作系统



层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

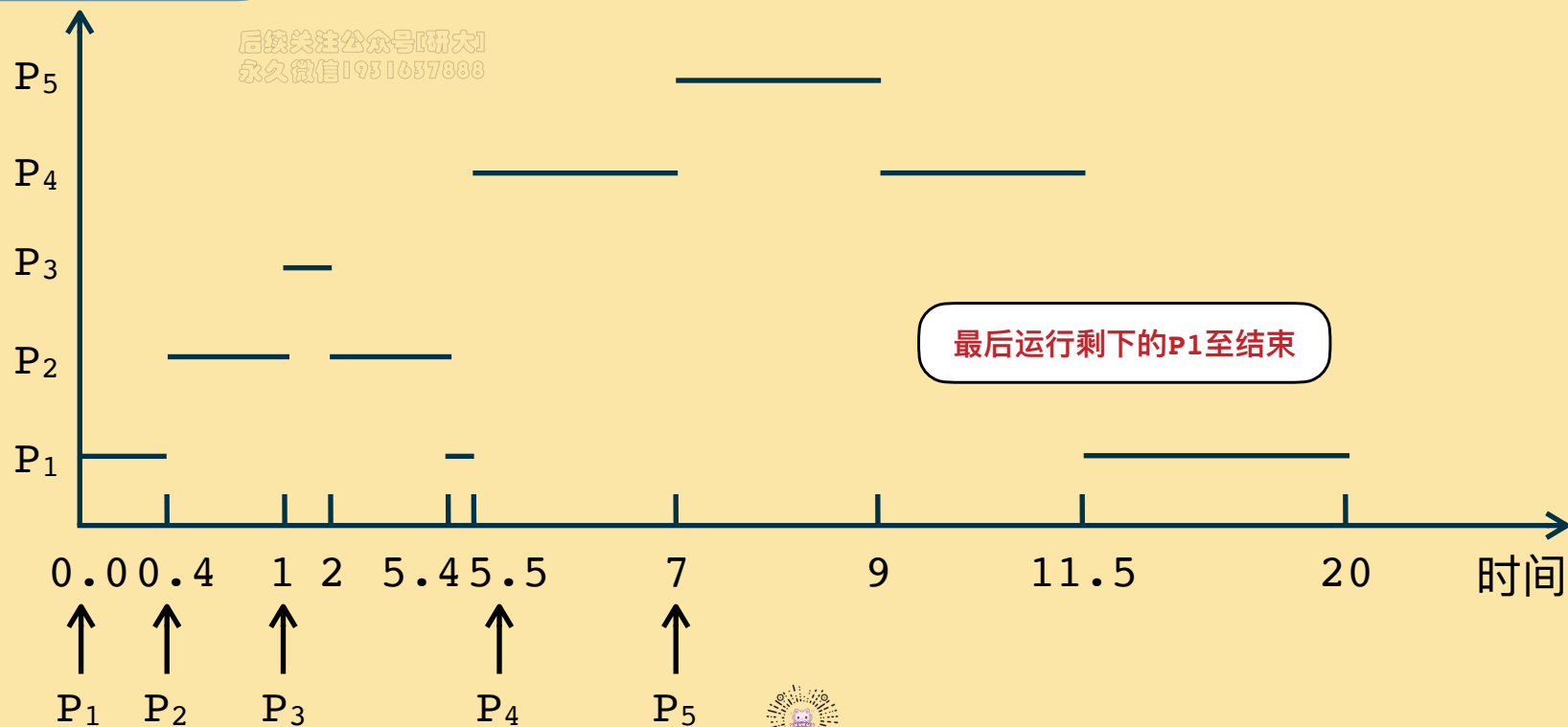
平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间





层次

目标

算法

多处理机调度

进程调度目标

为了比较调度算法的性能，设计了以下几种评价标准

平均周转时间

作业周转时间之和 / 作业数量

有以下进程需要调度执行

进程名	到达时间	运行时间
P ₁	0.0	9
P ₂	0.4	4
P ₃	1.0	1
P ₄	5.5	4
P ₅	7	2

若采用**抢占式短进程优先**调度算法
求这5进程的平均周转时间

后续关注公众号[研大]
永久微信1951657888

t=20	剩余运行时间	完成时间
P ₁	0	20
P ₂	0	5.4
P ₃	0	2
P ₄	0	11.5
P ₅	0	9

P₁周转时间: $20 - 0 = 20$

P₂周转时间: $5.4 - 0.4 = 5$

P₃周转时间: $2 - 1 = 1$

P₄周转时间: $11.5 - 5.5 = 6$

P₅周转时间: $9 - 7 = 2$

平均周转时间 = $(20 + 5 + 1 + 6 + 2) / 5 = 6.8$





层次

目标

算法

多处理机调度

处理机调度算法

先来先服务FCFS

短作业优先SJF

高响应比优先

优先级调度

时间片轮转

多级队列调度

多级反馈队列调度





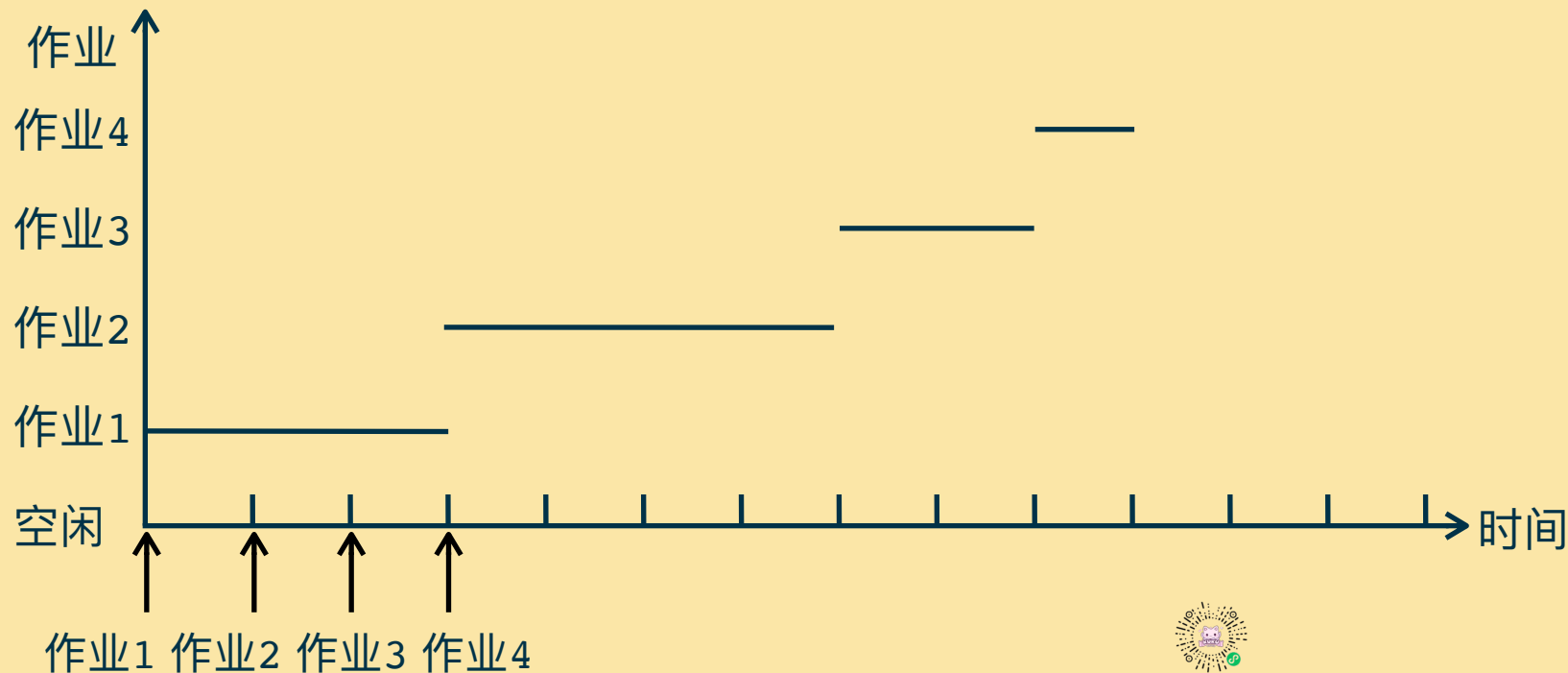
处理机调度算法

先来先服务FCFS

最简单的调度算法，既可以用在作业调度，也可以用在进程调度

每次选择最先进入队列的**作业/进程**，将它们**调入内存/分配CPU**

作业号	提交时间	运行时间	开始时间	等待时间	完成时间	周转时间	带权周转时间
1	0	3	0	0	3	3	1
2	1	4	3	2	7	6	6/4
3	2	2	7	5	9	7	7/2
4	3	1	9	6	10	7	7





处理机调度

层次

目标

算法

多处理机调度

处理机调度算法

先来先服务FCFS

最简单的调度算法，既可以用于作业调度，也可以用于进程调度
每次选择最先进入队列的**作业/进程**，将它们**调入内存/分配CPU**

按照作业/进程到达的先后次序进行调度

只能是非抢占式

对长作业有利，对短作业不利

有利于CPU繁忙型作业/进程，不利于IO繁忙型作业/进程





处理机调度算法

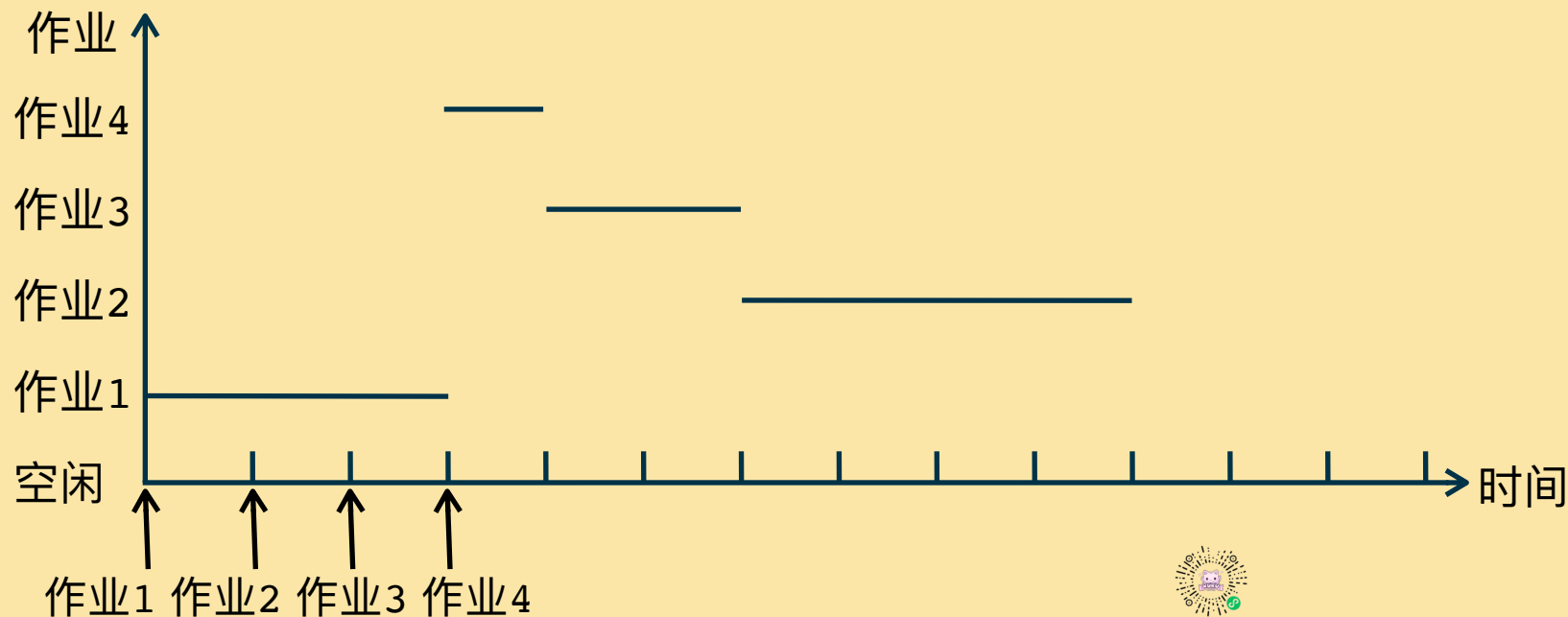
短作业优先SJF

(未说明则默认非抢占式)

针对短作业/进程优先调度的算法

它从后备队列/就绪队列中选择一个或几个估计运行时间最短的作业/进程，将它们调入内存运行

作业号	提交时间	运行时间	开始时间	等待时间	完成时间	周转时间	带权周转时间
1	0	3	0	0	3	3	1
2	1	4	6	5	10	9	9/4
3	2	2	4	2	6	4	2
4	3	1	3	0	4	1	1





处理机调度算法

短作业优先SJF

(未说明则默认非抢占式)

指对短作业/进程优先调度的算法

它从后备队列/就绪队列中选择一个或几个估计运行时间最短的作业/进程，将它们调入内存运行

作业/进程越短，优先级越高，越先被调度

后续关注公众号[研大]

公众号ID:10511657888

如果是抢占式SJF，作业/进程的剩余时间越短，优先级越高，越先被调度

平均等待时间和平均周转时间最优

对短作业有利，对长作业不利，甚至导致长作业饥饿

默认在估计运行时间相等时按照先后顺序服务



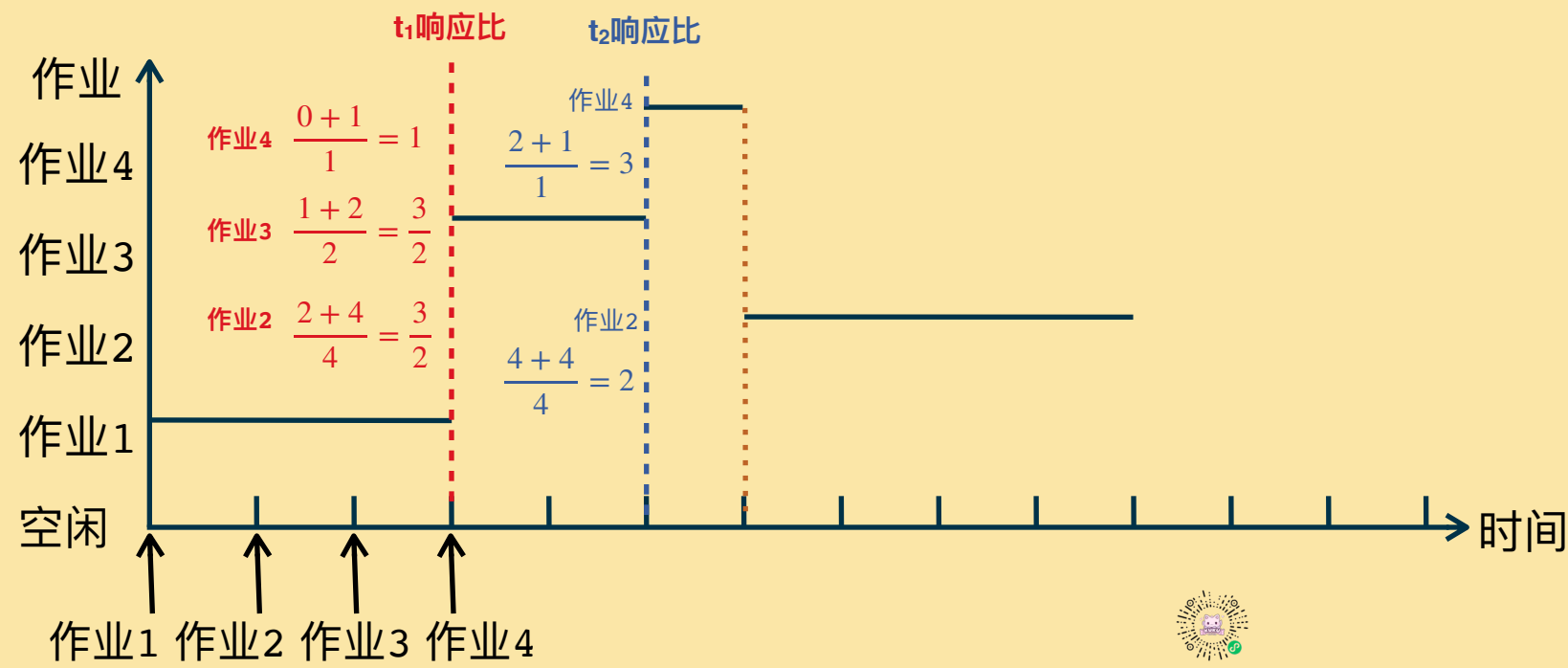
处理机调度算法

高响应比优先

响应比 = $\frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}}$

主要用于作业调度，同时考虑了等待时间和估计运行时间，每次调度时选择响应比最高的作业投入运行

作业号	提交时间	运行时间	开始时间	等待时间	完成时间	周转时间	带权周转时间
1	0	3	0	0	3	3	1
2	1	4	6	5	10	9	9/4
3	2	2	3	1	5	3	3/2
4	3	1	5	2	6	3	3





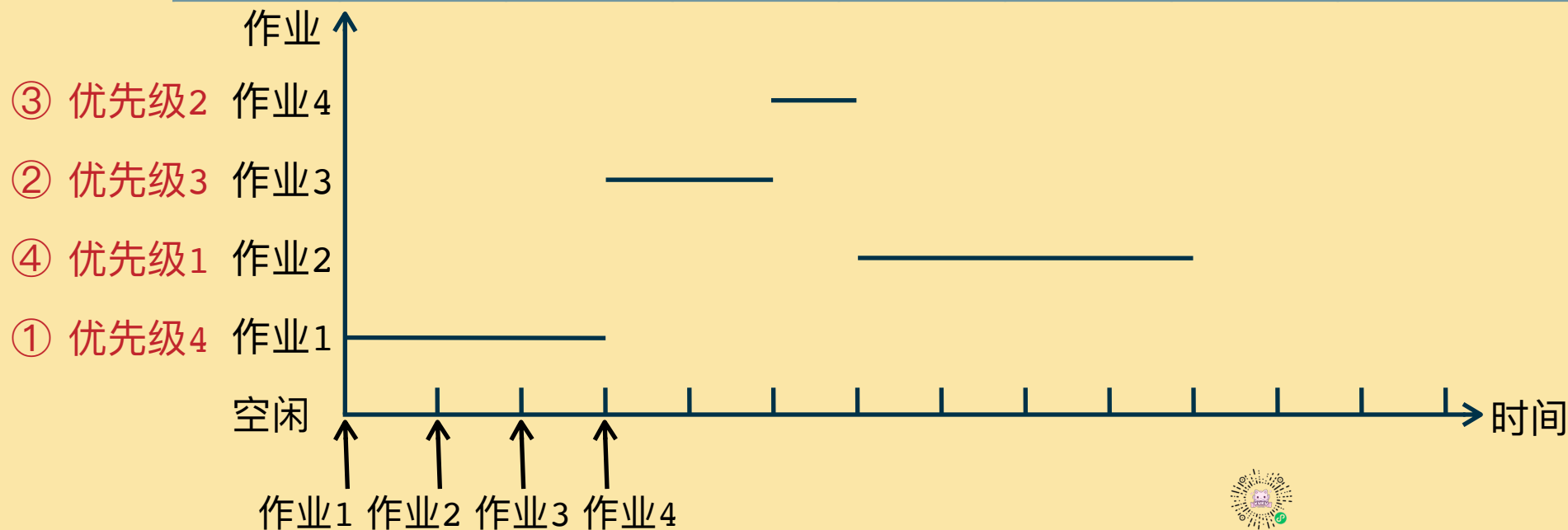
处理机调度算法

优先级调度

既可用于作业调度，又可用于进程调度

该算法中的优先级用于描述作业的紧迫程度。每次选**优先级最高**的作业/进程放入就绪队列或运行

作业号	提交时间	运行时间	开始时间	等待时间	完成时间	周转时间	带权周转时间
1	0	3	0	0	3	3	1
2	1	4	6	5	10	9	9/4
3	2	2	3	1	5	3	3/2
4	3	1	5	2	6	3	3





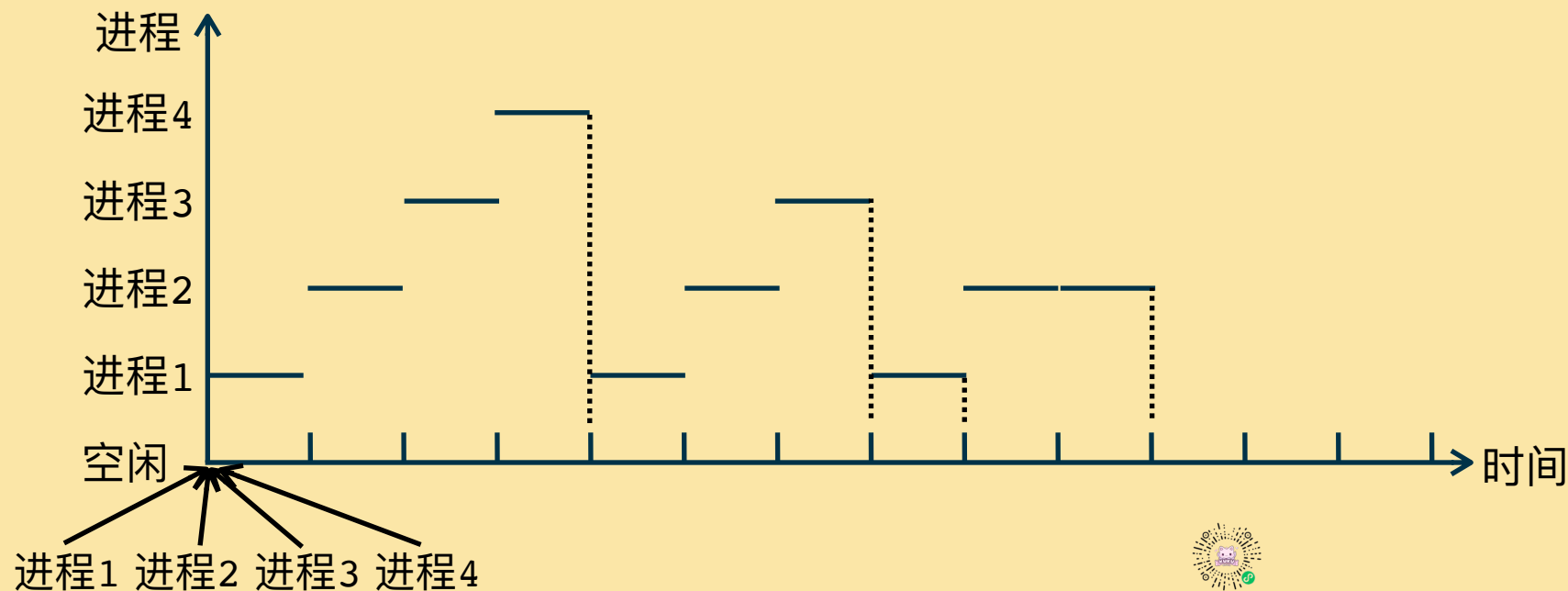
处理机调度算法

时间片轮转

主要用在分时系统，所有就绪进程排成就绪队列

OS每隔一段时间就产生一次时钟中断，调度进程，将CPU分配给就绪队列队首进程，令其执行一个时间片

进程号	提交时间	运行时间	开始时间	等待时间	完成时间	周转时间	带权周转时间
1	0	3	0	5	8	8	8/3
2	0	4	1	6	10	10	10/4
3	0	2	2	2	7	7	7/2
4	0	1	3	3	4	4	4





处理机调度算法

多级队列调度

设置多个队列，将不同类型的进程分配到不同的就绪队列，每个队列实行不同的调度算法

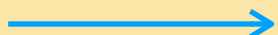
最高优先级



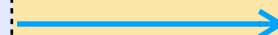
系统进程
优先级调度算法



后浪关注公众号【码农六六】
公众号ID: 10511657888



交互进程
时间片轮转调度算法



最低优先级



批处理进程
先来先服务调度算法



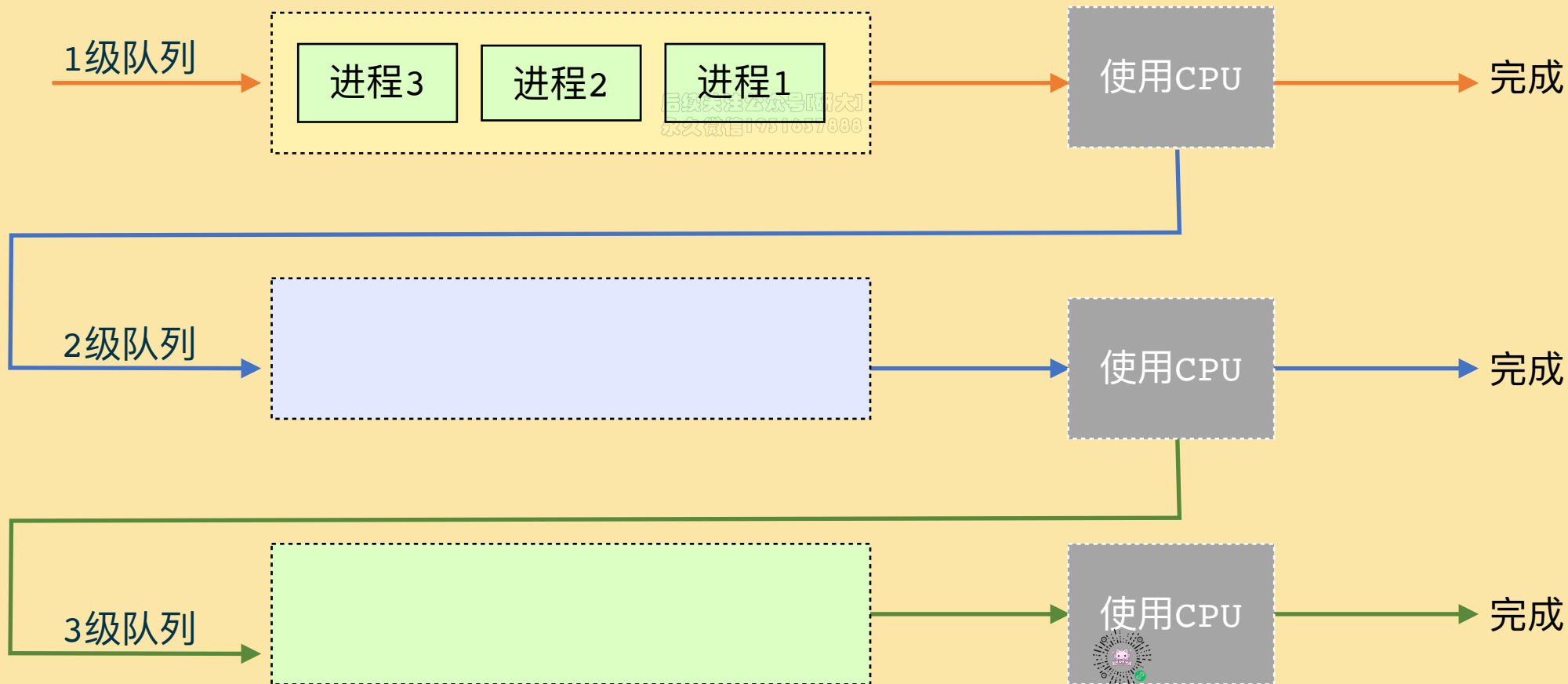


处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法，设置多个就绪队列，为每个队列赋予不同优先级
赋予各个队列的进程运行时间片不同；每个队列采用FCFS算法。

若在时间片内完成，则撤离系统
否则转入2级队列



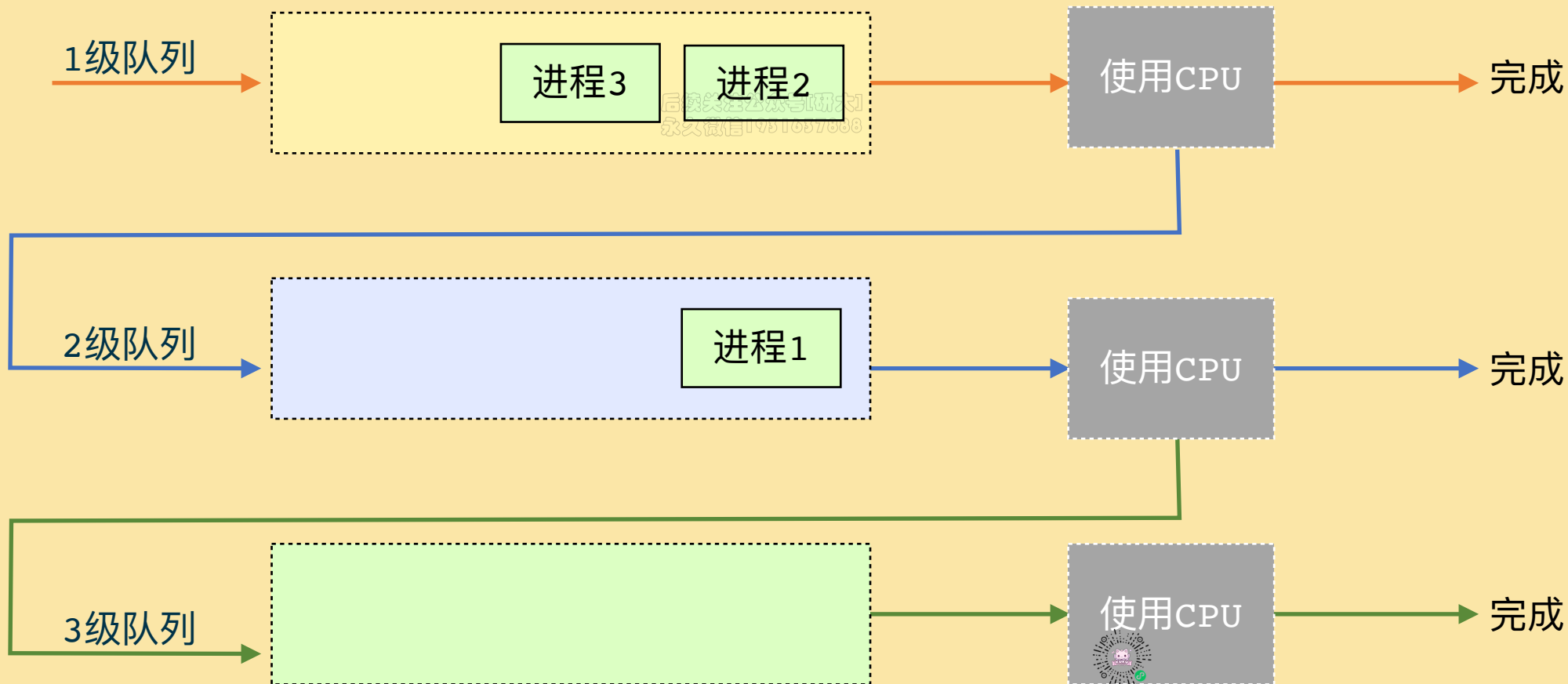


处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法，设置多个就绪队列，为每个队列赋予不同优先级
赋予各个队列的进程运行时间片不同；每个队列采用FCFS算法。

若在时间片内完成，则撤离系统
否则转入2级队列



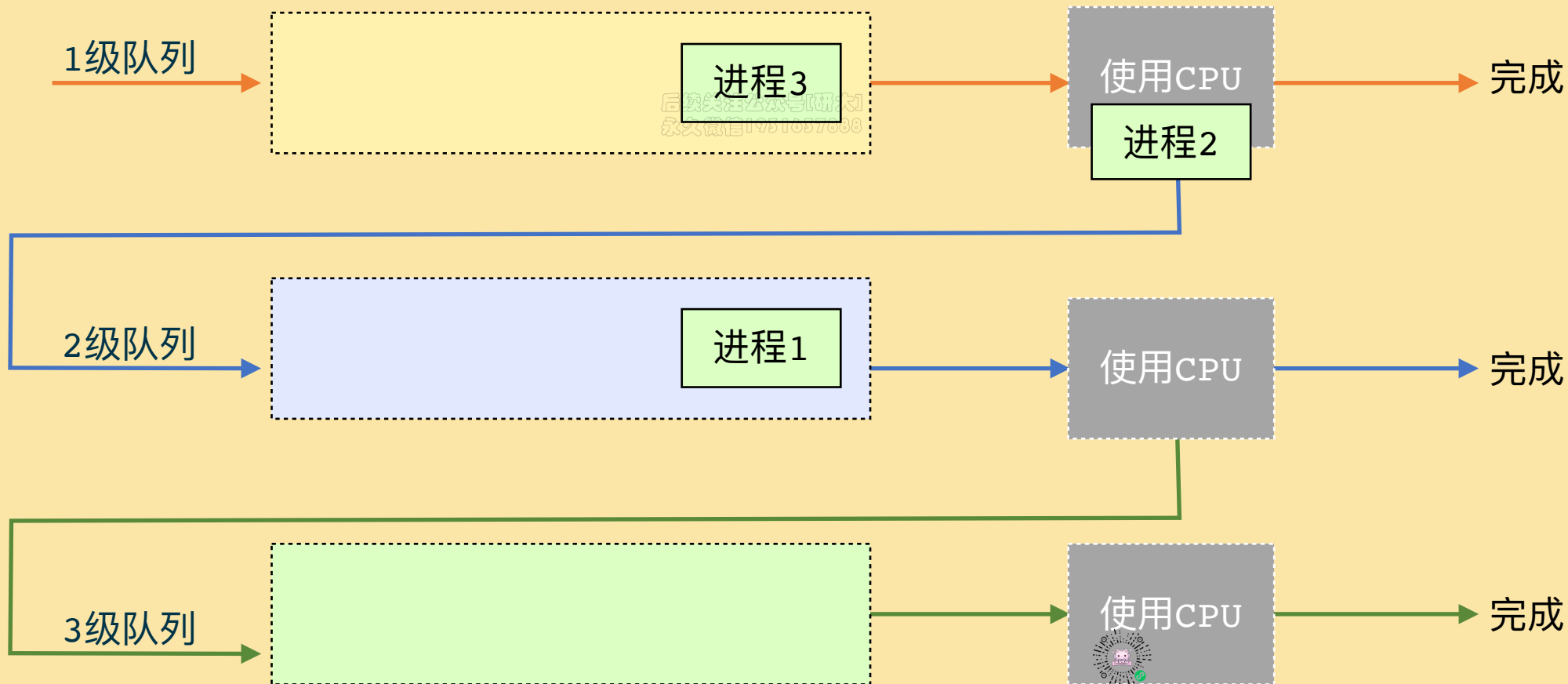


处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法，设置多个就绪队列，为每个队列赋予不同优先级
赋予各个队列的进程运行时间片不同；每个队列采用FCFS算法。

若在时间片内完成，则撤离系统
否则转入2级队列



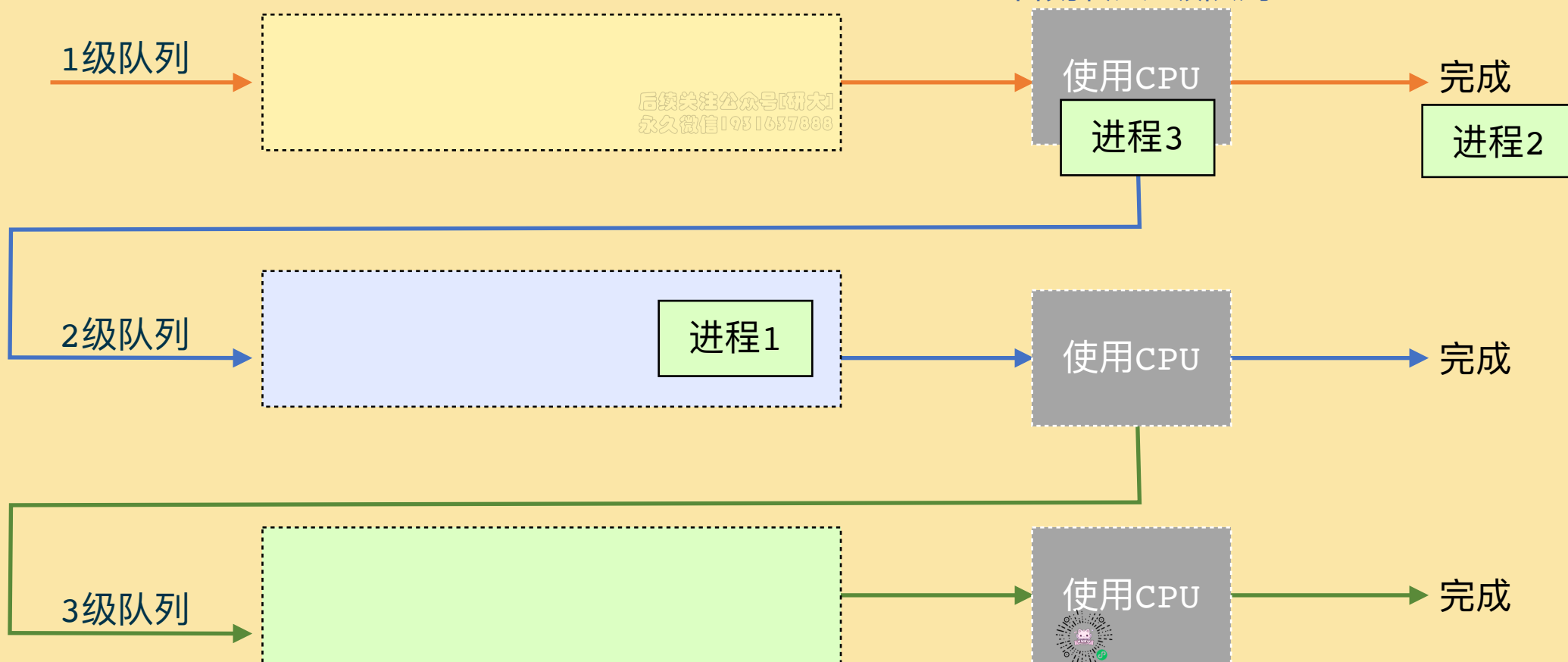


处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法，设置多个就绪队列，为每个队列赋予不同优先级
赋予各个队列的进程运行时间片不同；每个队列采用FCFS算法。

若在时间片内完成，则撤离系统
否则转入2级队列



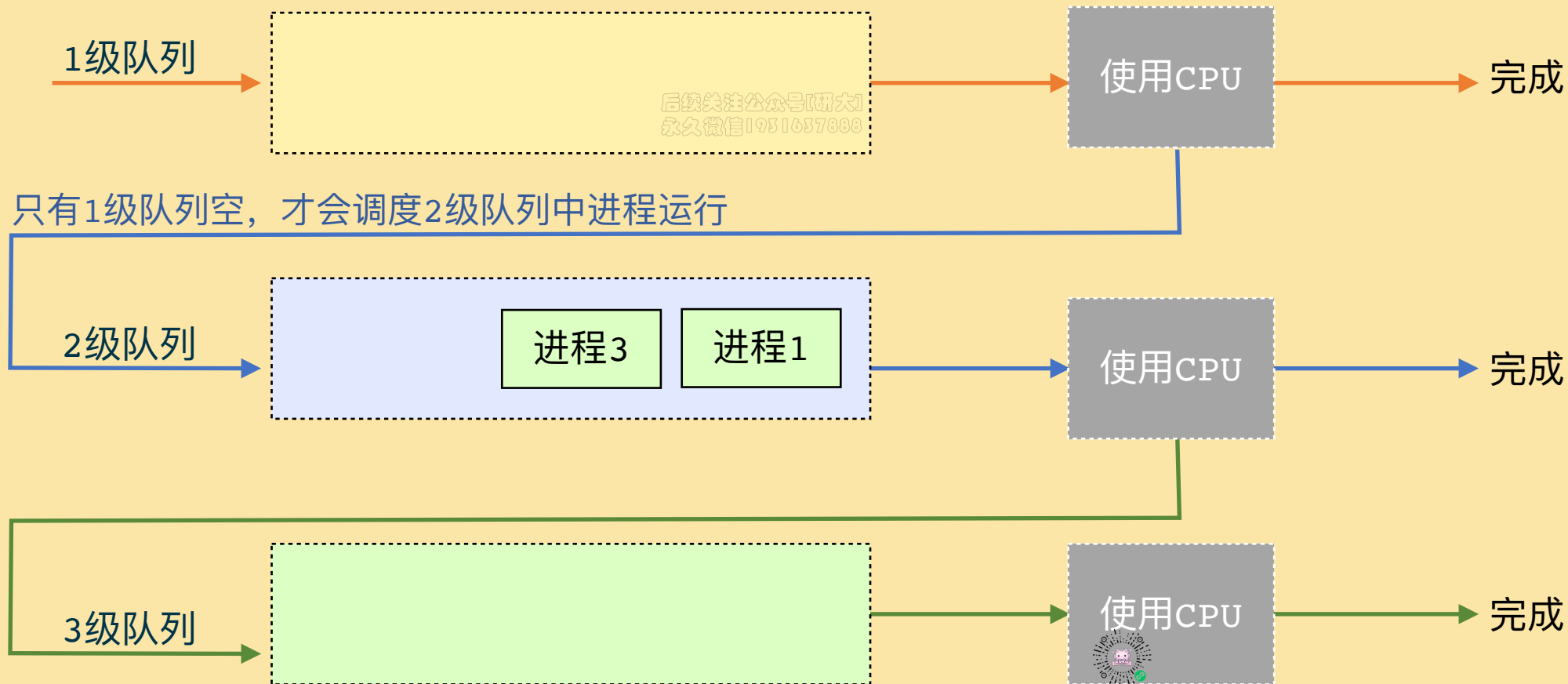


处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法，设置多个就绪队列，为每个队列赋予不同优先级
赋予各个队列的进程运行时间片不同；每个队列采用FCFS算法。

若在时间片内完成，则撤离系统
否则转入2级队列



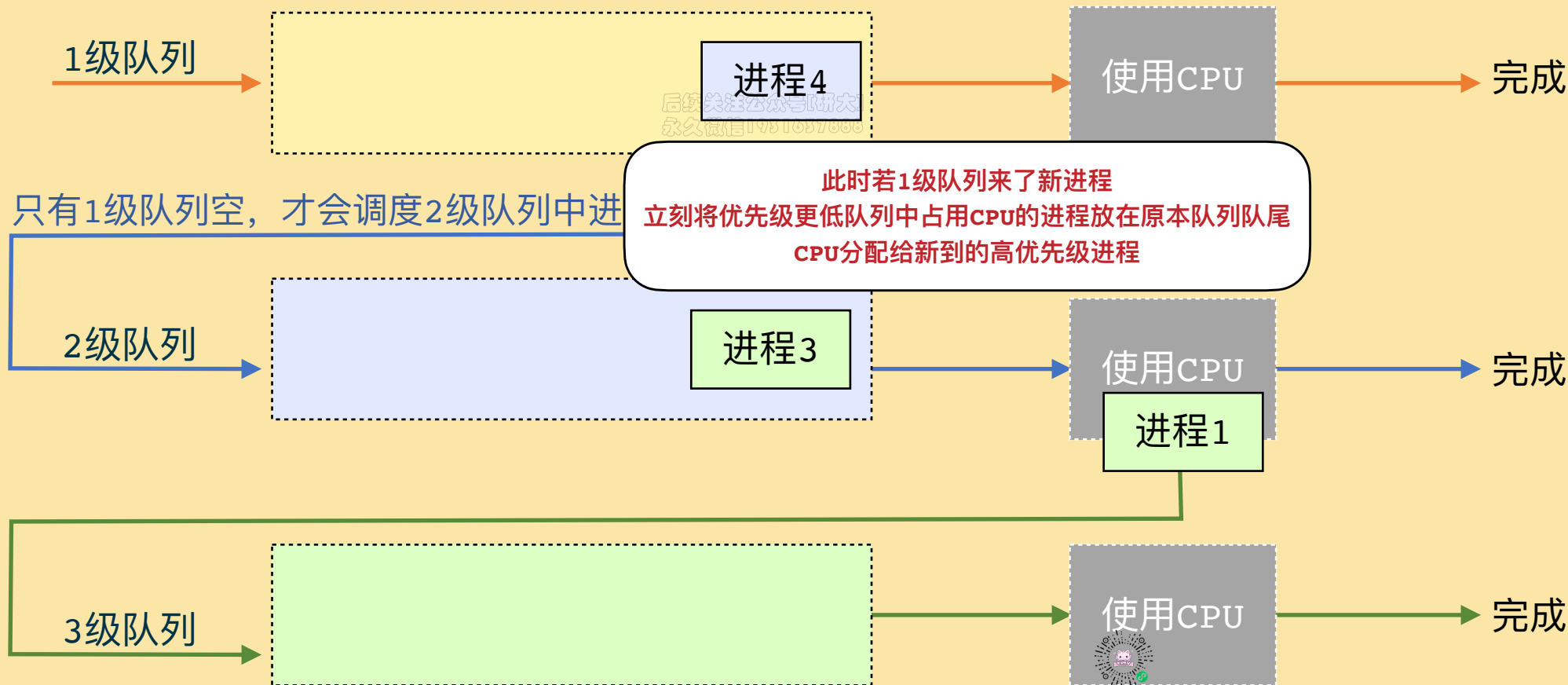


处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法，设置多个就绪队列，为每个队列赋予不同优先级
赋予各个队列的进程运行时间片不同；每个队列采用FCFS算法。

若在时间片内完成，则撤离系统
否则转入2级队列



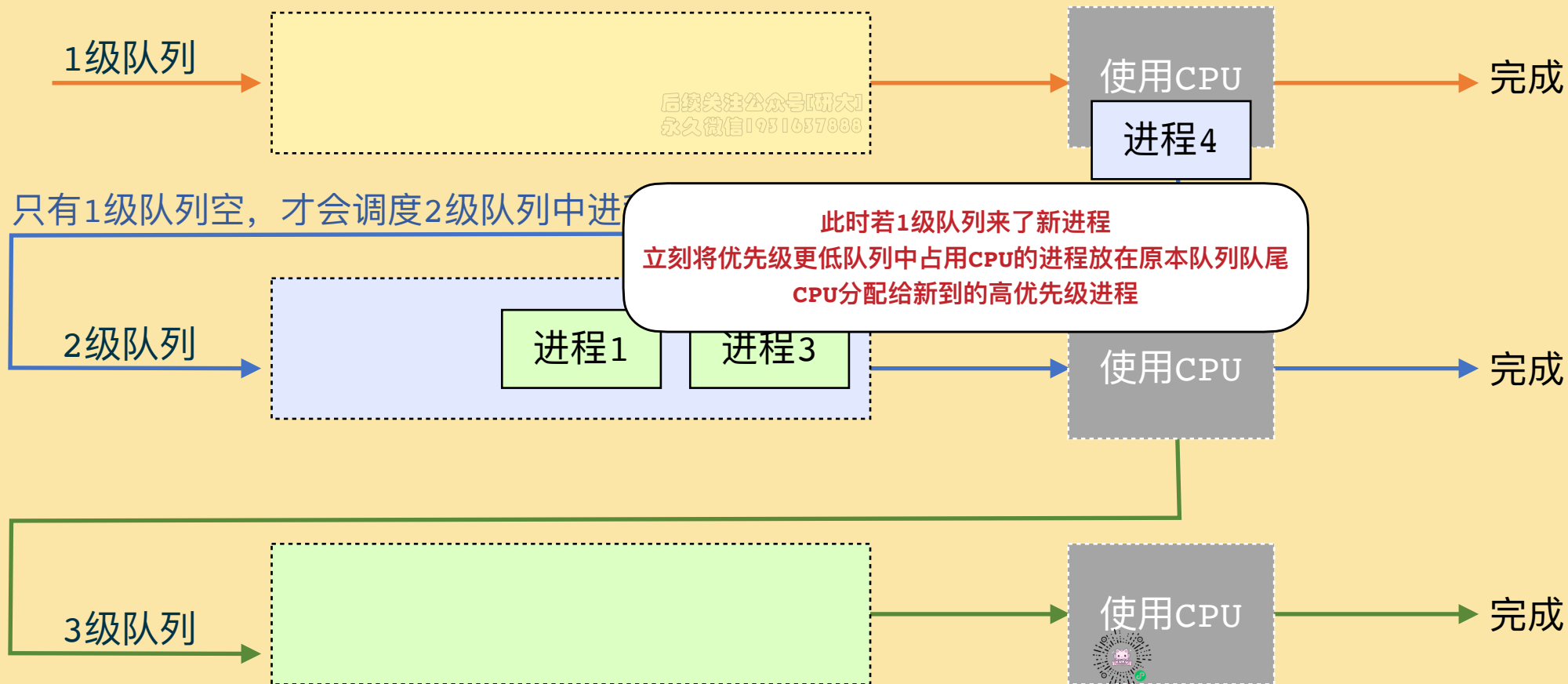


处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法，设置多个就绪队列，为每个队列赋予不同优先级
赋予各个队列的进程运行时间片不同；每个队列采用FCFS算法。

若在时间片内完成，则撤离系统
否则转入2级队列



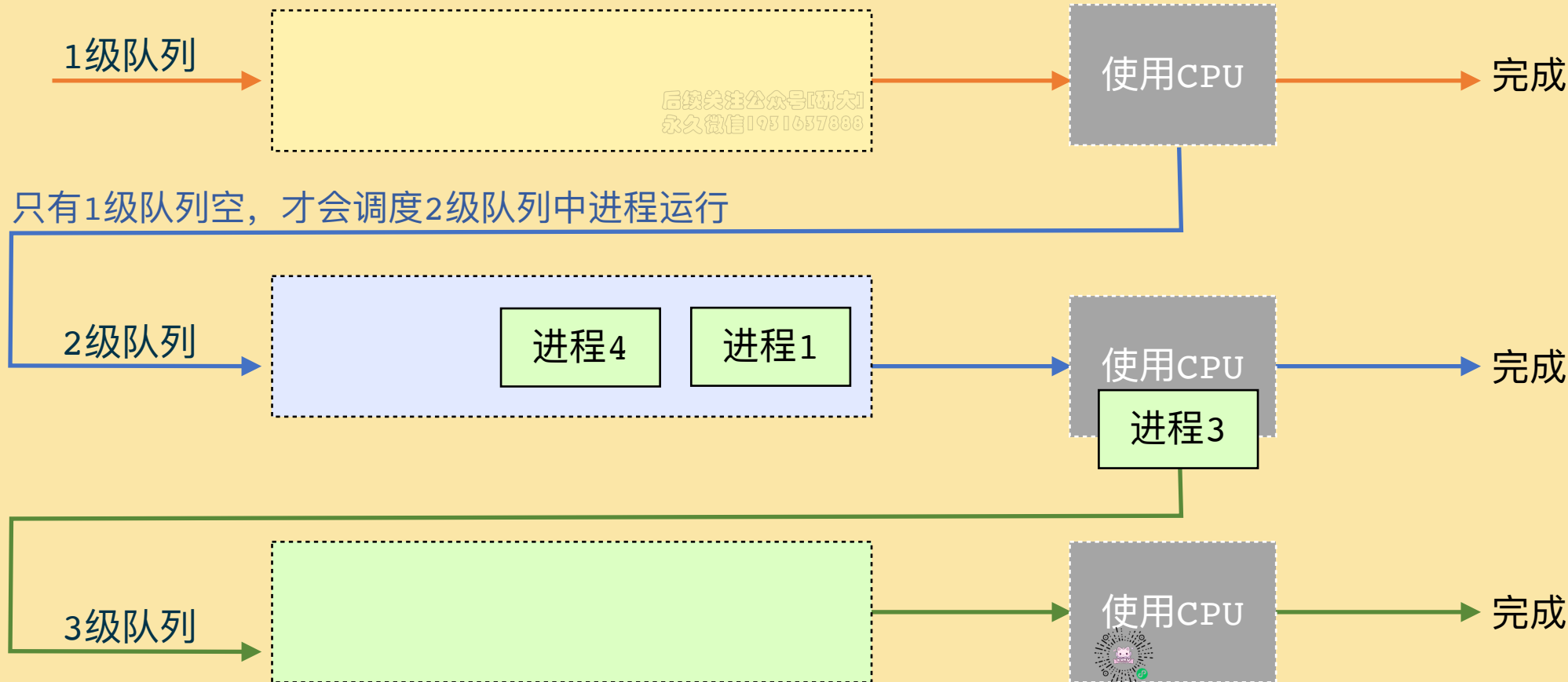


处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法，设置多个就绪队列，为每个队列赋予不同优先级
赋予各个队列的进程运行时间片不同；每个队列采用FCFS算法。

若在时间片内完成，则撤离系统
否则转入2级队列



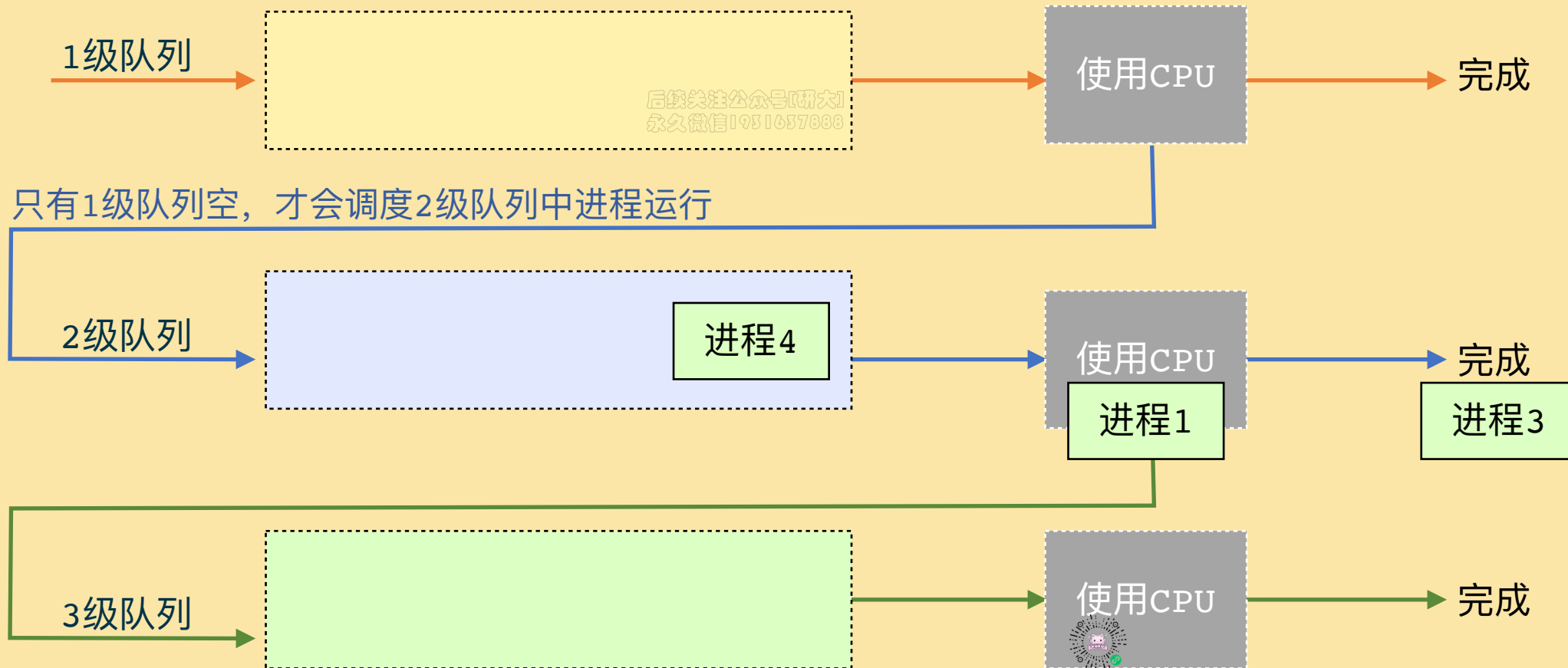


处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法，设置多个就绪队列，为每个队列赋予不同优先级
赋予各个队列的进程运行时间片不同；每个队列采用FCFS算法。

若在时间片内完成，则撤离系统
否则转入2级队列



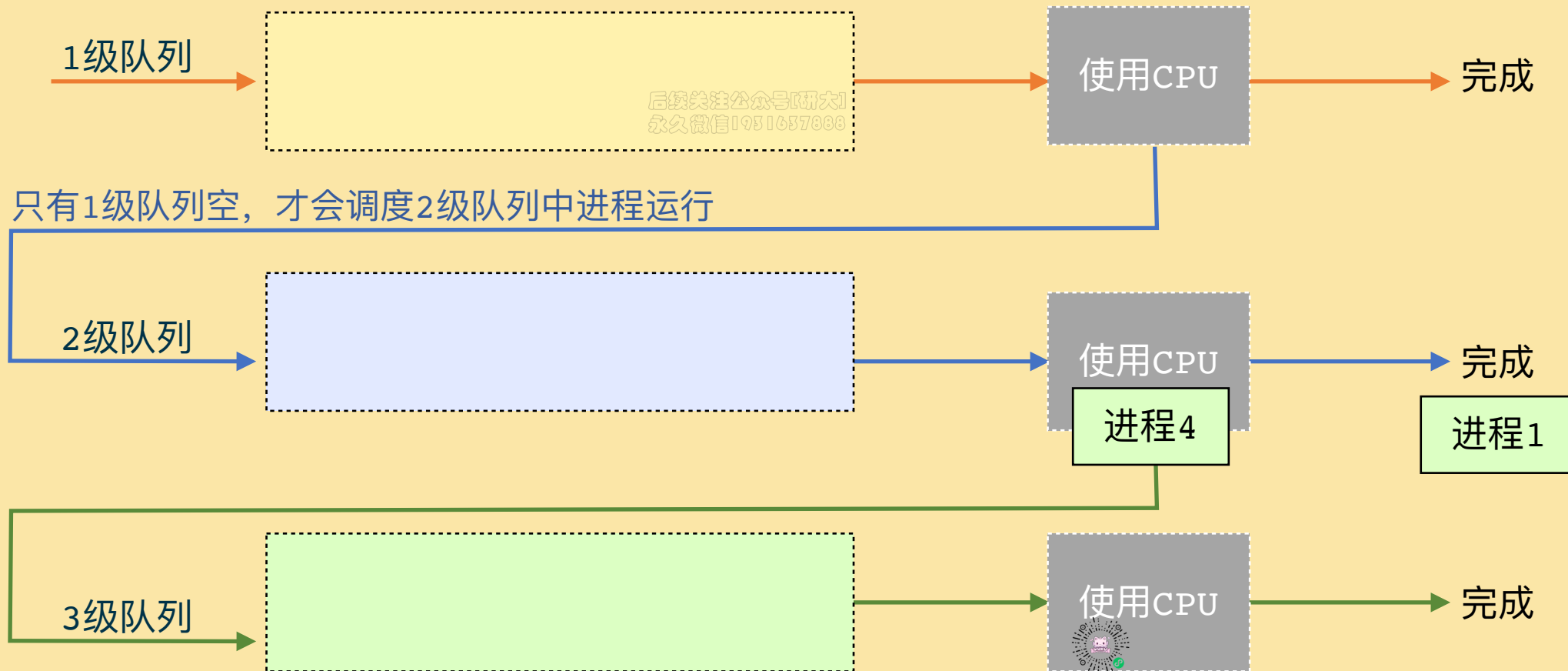


处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法，设置多个就绪队列，为每个队列赋予不同优先级
赋予各个队列的进程运行时间片不同；每个队列采用FCFS算法。

若在时间片内完成，则撤离系统
否则转入2级队列



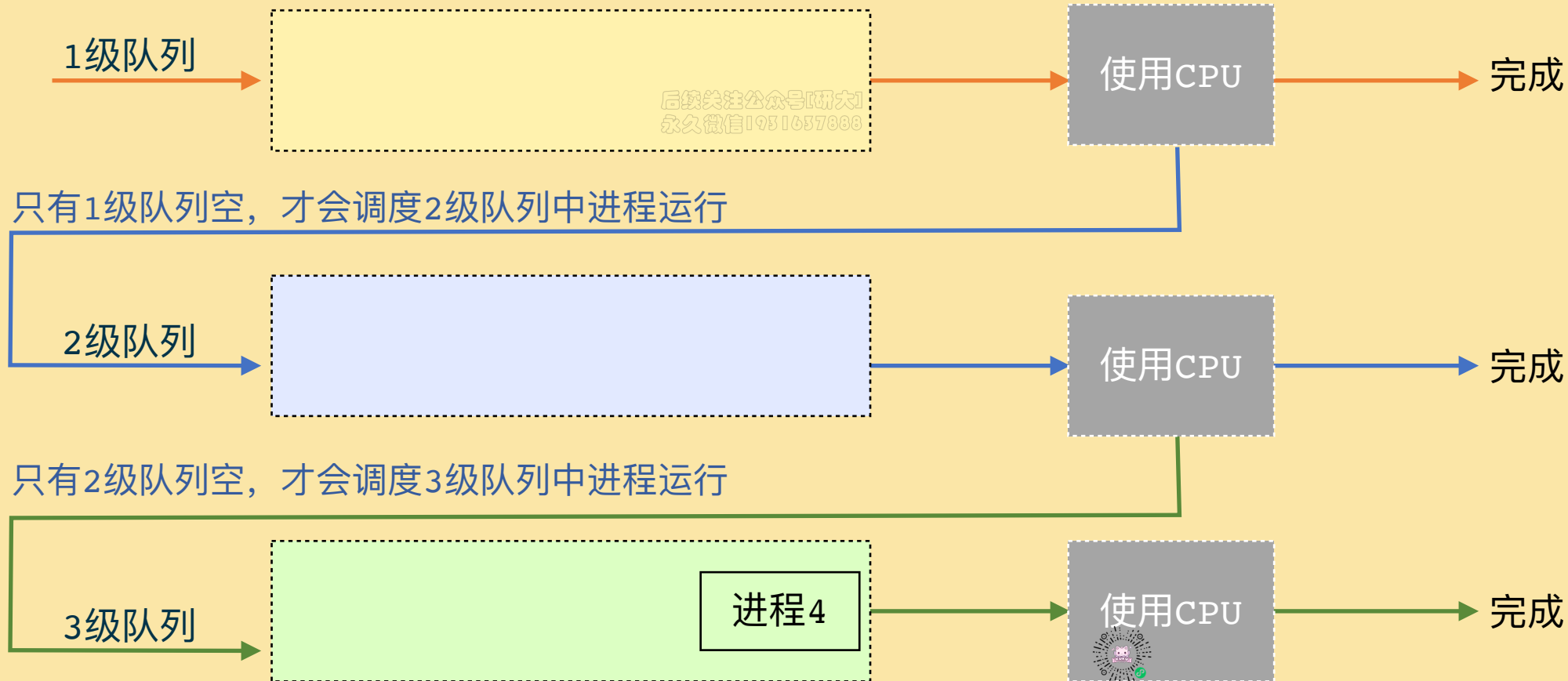


处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法，设置多个就绪队列，为每个队列赋予不同优先级
赋予各个队列的进程运行时间片不同；每个队列采用FCFS算法。

若在时间片内完成，则撤离系统
否则转入2级队列



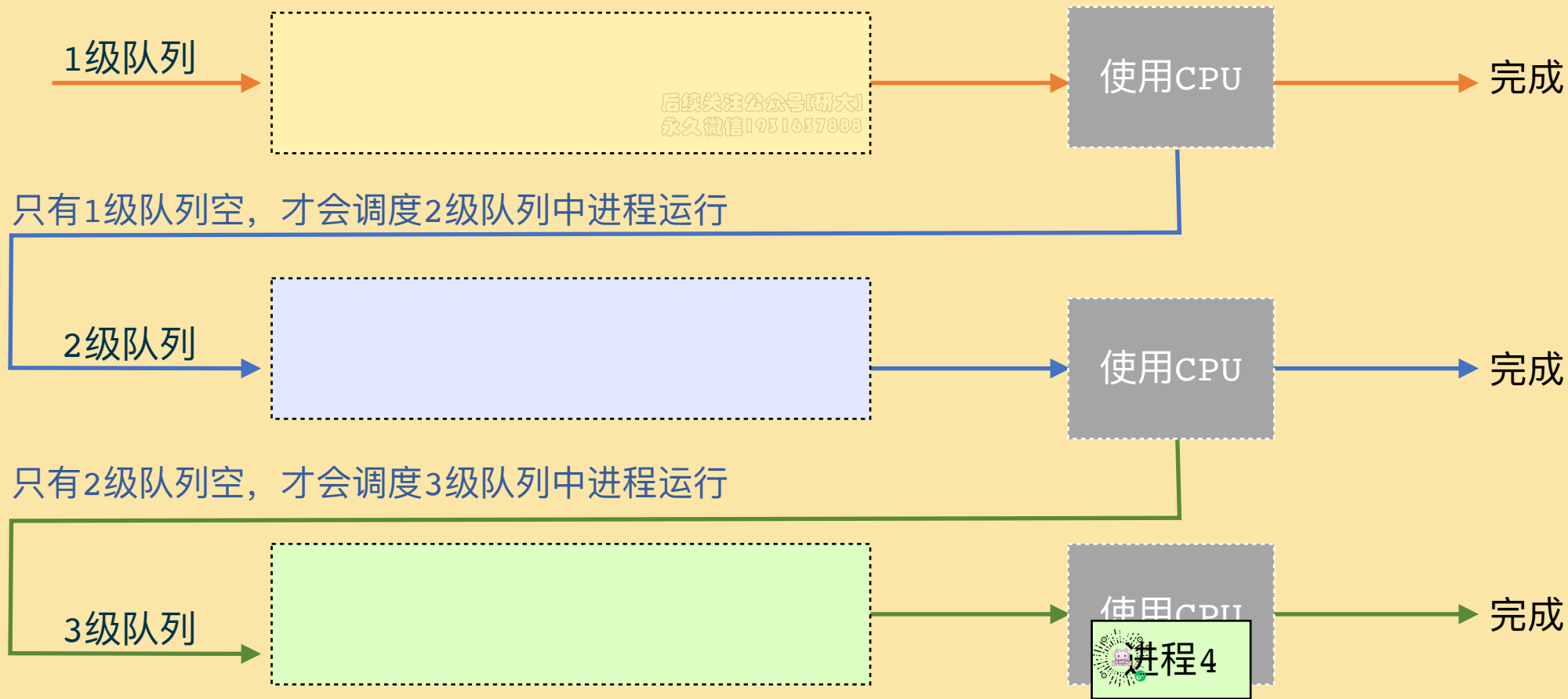


处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法，设置多个就绪队列，为每个队列赋予不同优先级
赋予各个队列的进程运行时间片不同；每个队列采用FCFS算法。

若在时间片内完成，则撤离系统
否则转入2级队列





处理机调度算法

多级反馈队列调度

结合了时间片调度和优先级调度算法

1. 设置多个就绪队列，每个队列赋予不同优先级，如1级队列最高优先级，3级最低优先级

2. 赋予各个队列的进程运行时间片大小各不相同。优先级越高的队列，时间片越小

3. 每个队列都采用FCFS，新进程进入内存，首先放在1级队列末尾等待调度
轮到该进程执行时，若一个时间片内执行完成，则撤离系统；若未完成，则转入下一级队列末尾等待调度

4. 只有1级队列为空，才调度第2级队列中的进程运行；仅当1~i-1级队列均为空时，才调度第i级队列中的进程运行





层次

目标

算法

多处理机调度

处理机调度算法

先来先服务FCFS

短作业优先SJF

高响应比优先

优先级调度

时间片轮转

多级队列调度

多级反馈队列调度





处理机调度

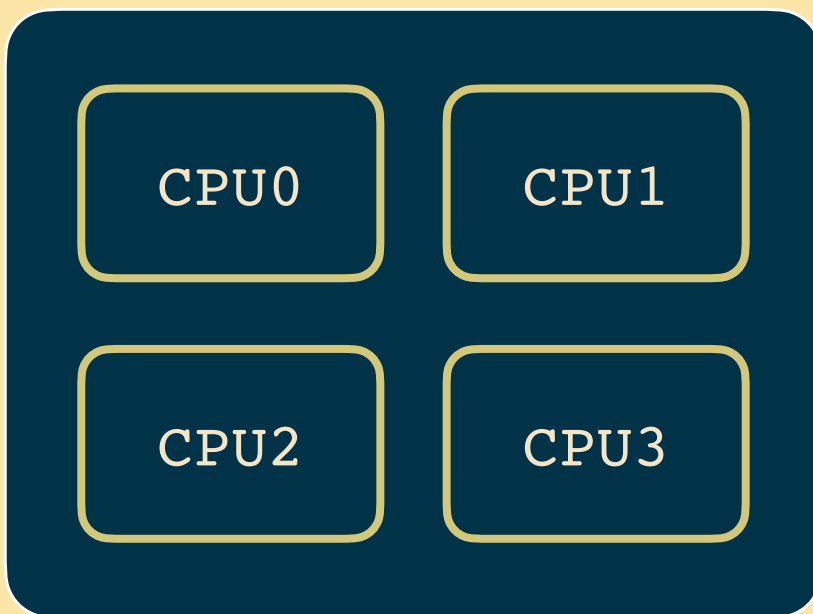
层次

目标

算法

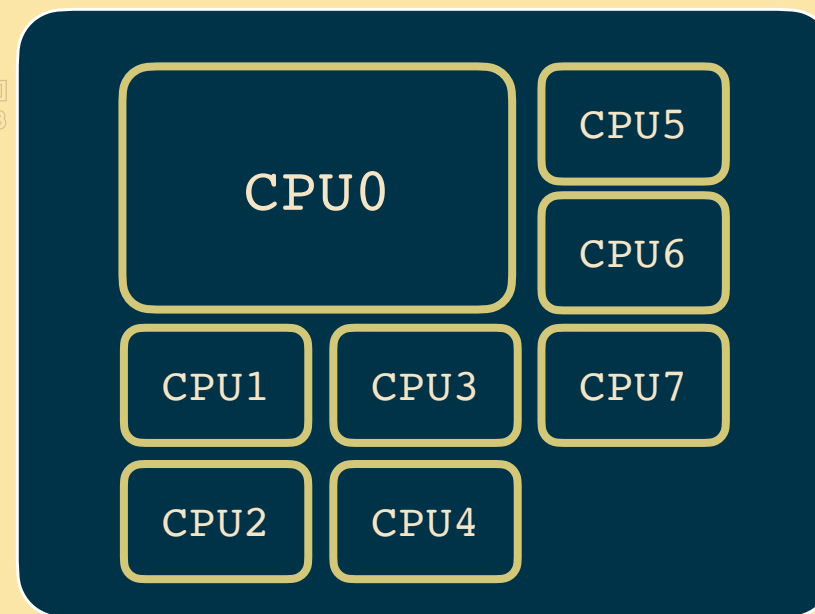
多处理机调度

多处理机系统



对称多处理机

后续关注公众号「研大」
永久微信1951657888



非对称多处理机





处理机调度

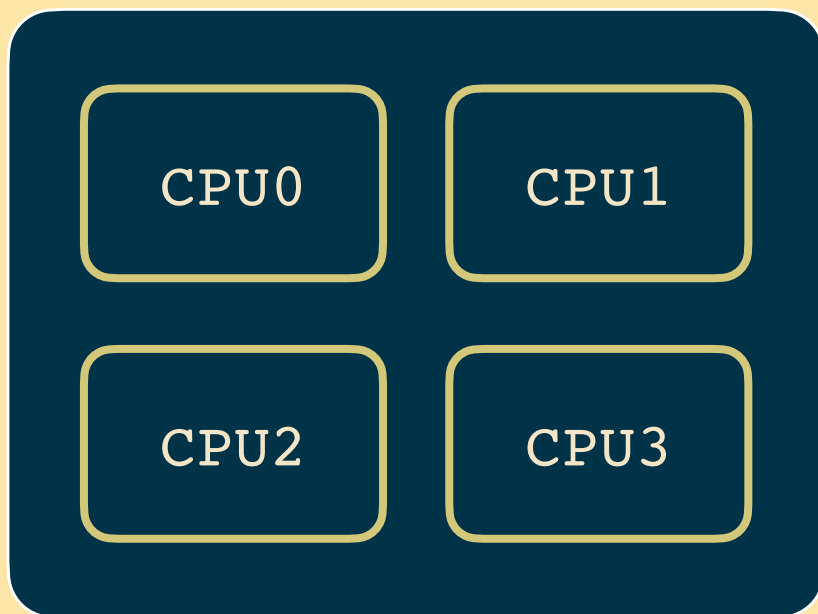
层次

目标

算法

多处理机调度

多处理机系统



对称多处理机

后续关注公众号[研大]
永久微信1951657888

对称多处理机 (Symmetric MultiProcessing, SMP)

所有CPU地位对等, 均可参与调度

调度程序可将任意就绪进程分配给任意空闲 CPU

主要学习的内容





处理机调度

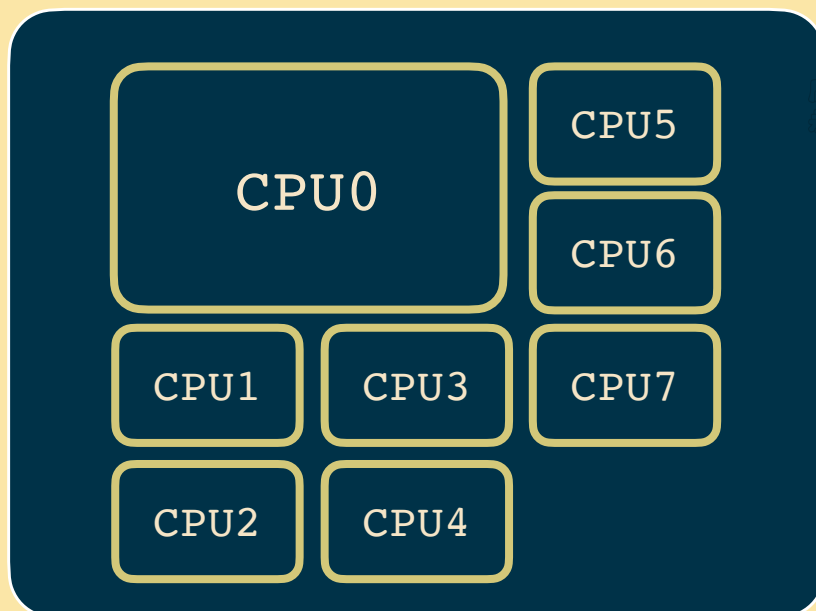
层次

目标

算法

多处理机调度

多处理机系统



非对称多处理机

一个主CPU负责系统管理和进程调度

内核通常运行在主CPU上

其他从CPU只负责执行主CPU分配的任务，不参与调度

从CPU空闲时向主CPU请求任务

主CPU从全局就绪队列中选择进程并分配

优点是实现简单、管理方便

缺点是调度任务集中在主 CPU，调度开销较大
了解即可





处理机调度

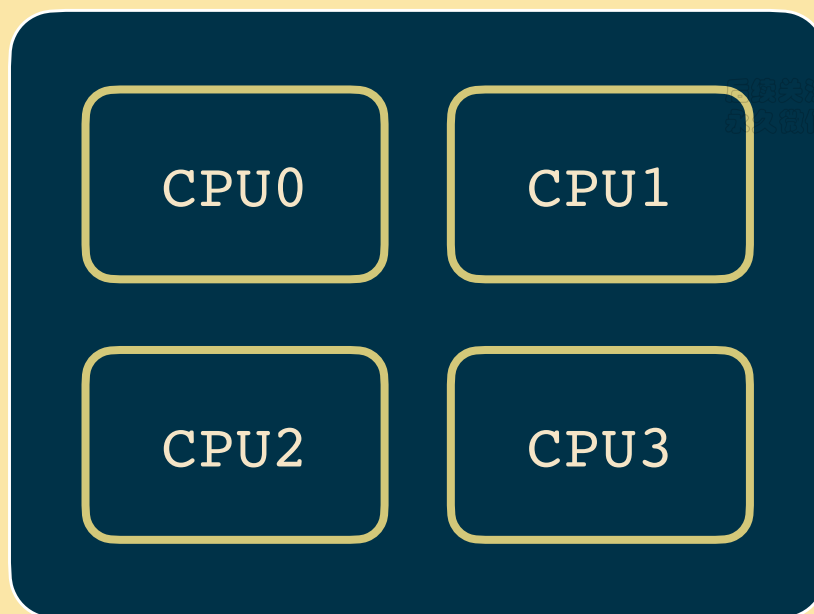
层次

目标

算法

多处理机调度

多处理机系统



对称多处理机

缓存亲和性

负载均衡





处理机调度

层次

目标

算法

多处理机调度

多处理机系统

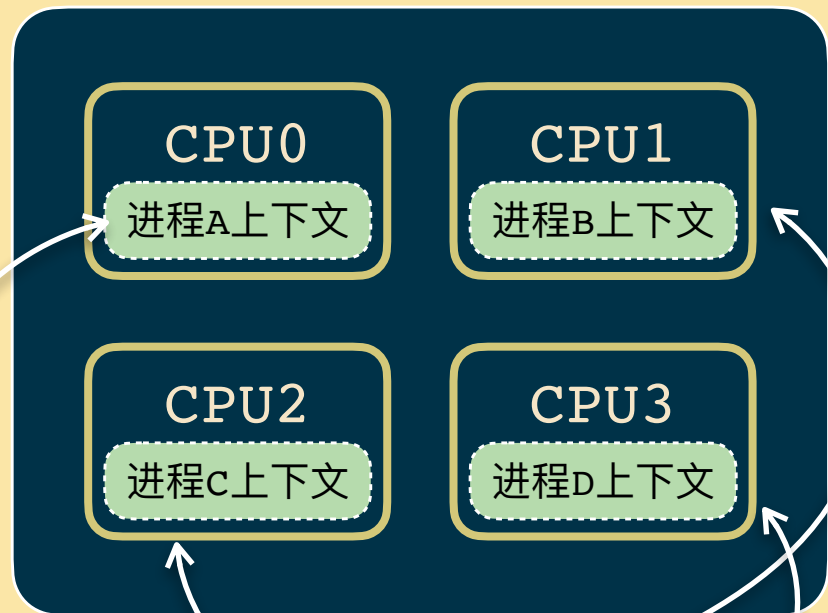
缓存亲和性

负载均衡

后续关注公众号「研大」
永久微信1951657888

一个进程在某个CPU上运行时
会在该CPU的缓存 (Cache, TLB) 中维护许多状态
下次该进程在相同CPU上运行时
由于缓存中的数据而执行得更快
相反, 在不同的CPU上执行, 会由于需要重新加载数据而很慢
因此多处理器调度应该考虑到这种缓存亲和性
并尽可能将进程保持在同一个CPU上

把进程分配给有上下文的
处理器更高效



对称多处理机

就绪队列

进程A

进程B

进程C

进程D



处理机调度

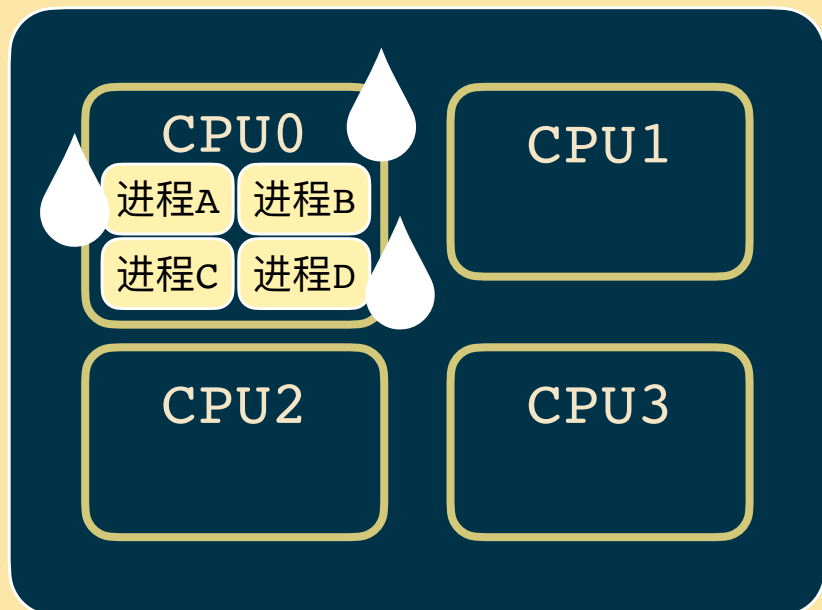
层次

目标

算法

多处理机调度

多处理机系统



对称多处理机

后续关注公众号
永久微信1951657888

缓存亲和性

负载均衡

要尽量让每个处理器都负载均衡
从而最大化发挥多处理器的优势

避免发生“一核有难，七核围观”的情况





层次

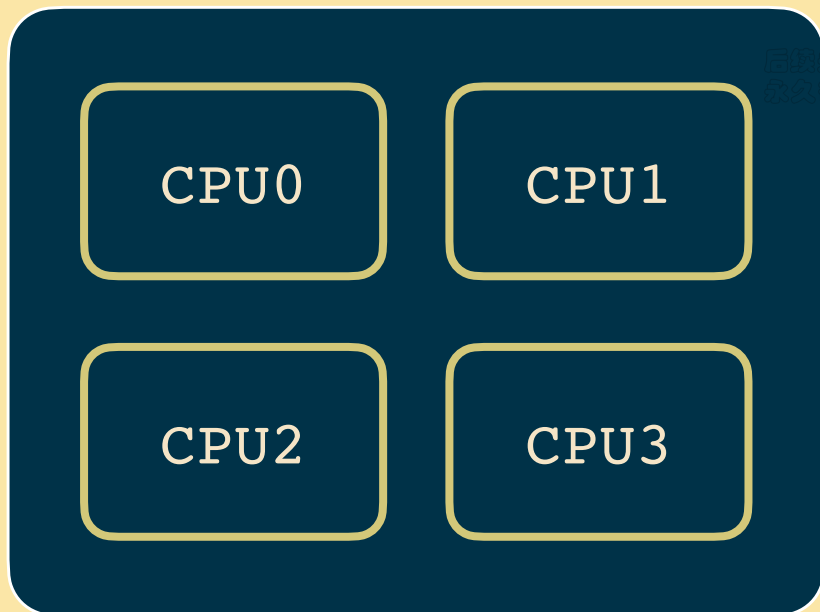
目标

算法

多处理机调度

动态分配方式

为了防止多个处理器忙闲不均，在系统中只设置一个公共就绪队列
分配进程时，可将进程分配到任何一个处理器上



扫码关注公众号「研大」
微信: 1951657888

就绪队列

进程A

进程B

进程C

进程D





层次

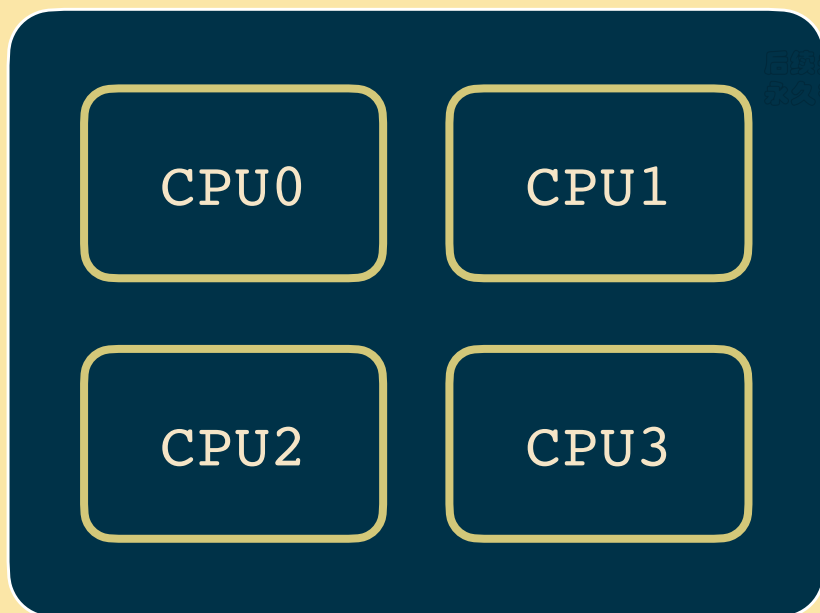
目标

算法

多处理机调度

动态分配方式

为了防止多个处理器忙闲不均，在系统中只设置一个公共就绪队列
分配进程时，可将进程分配到任何一个处理器上



就绪队列

进程A

进程B

进程C

进程D

软亲和：调度程序尽量将一个进程保持在某个 CPU 上运行
但这个进程也可以迁移到其他CPU上

硬亲和：指由用户进程通过系统调用
主动请求系统将其绑定到固定的 CPU 上



处理机调度



层次

目标

算法

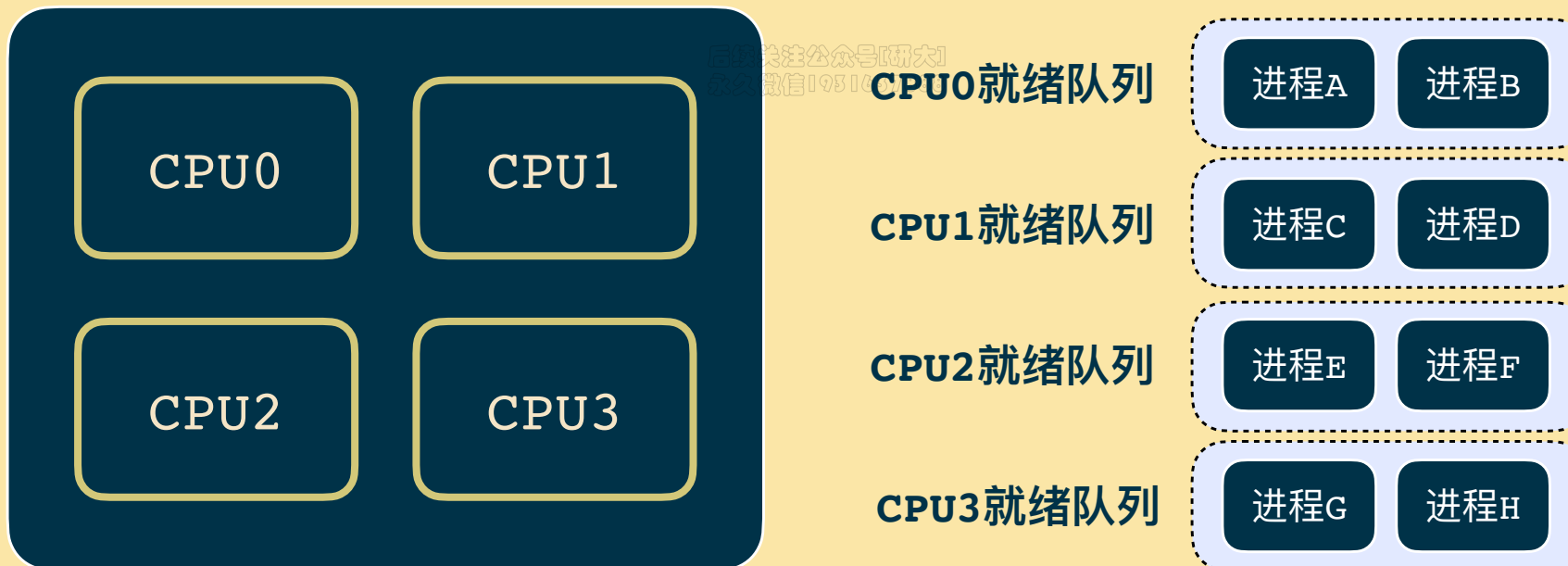
多处理机调度

静态分配方式

指进程从开始执行到执行完成，都被固定地分配到一个处理器上运行

每一个处理机需要设置一专用的就绪队列

就绪队列中的进程先后被分配到处理机上执行



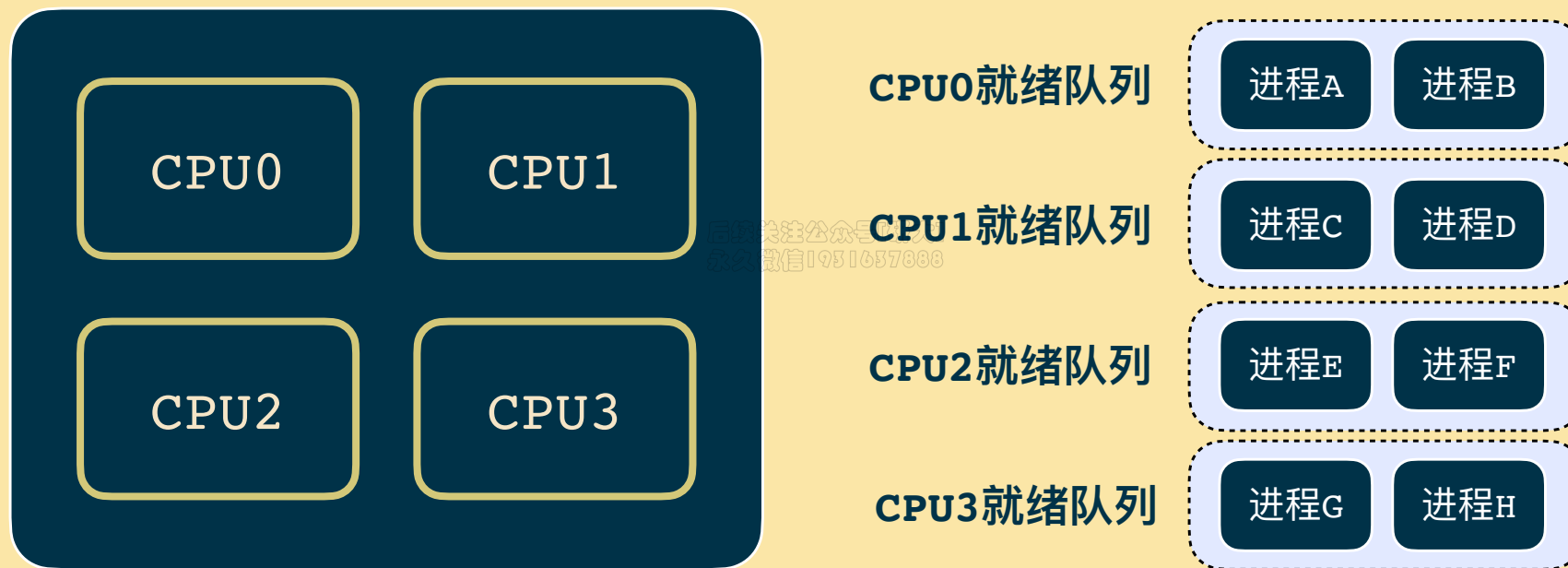


静态分配方式

指进程从开始执行到执行完成，都被固定地分配到一个处理器上运行

每一个处理机需要设置一专用的就绪队列

就绪队列中的进程先后被分配到处理机上执行



推迁移:指由一个特定的系统程序周期性地检查各CPU的负载
若发现不平衡，则从超载CPU 的就绪队列中“推”出部分进程迁移到空闲CPU的就绪队列

拉迁移:指当某个 CPU 负载很低时
主动从超载CPU 的就绪队列中“拉”取部分进程到自己的就绪队列